

Database systems ITCS2322

Chapter 10 : Transaction Management and Concurrency Control

اعداد : دكتور محمد عبد اللطيف راضي

Learning Objectives

- In this chapter, students will learn:
 - About database transactions and their properties
 - What concurrency control is and what role it plays in maintaining the database's integrity
 - What locking methods are and how they work

Learning Objectives

- In this chapter, students will learn:
 - How stamping methods are used for concurrency control
 - How optimistic methods are used for concurrency control
 - How database recovery management is used to maintain database integrity

Chapter 5: Advanced Data Modeling

Table of contents

- What Is a Transaction?
- Concurrency Control Concurrency Control with Locking
- Concurrency Control with Time Stamping Methods
- Concurrency Control with Optimistic Methods
- ANSI Levels of Transaction Isolation
- Database Recovery Management

Transaction

- Logical unit of work that must be entirely completed or aborted
- Consists of:
 - SELECT statement
 - Series of related UPDATE statements
 - Series of INSERT statements
 - Combination of SELECT, UPDATE, and INSERT statements

Transaction

- **Consistent database state:** All data integrity constraints are satisfied
 - Must begin with the database in a known consistent state to ensure consistency
- Formed by two or more database requests
 - **Database requests:** Equivalent of a single SQL statement in an application program or transaction
- Consists of a single SQL statement or a collection of related SQL statements

Evaluating Transaction Results

Transaction can be simple SQL code:

```
SELECT      CUST_NUMBER, CUST_BALANCE
FROM        CUSTOMER
WHERE       CUST_NUMBER = 10016;
```

Transaction may consist of a single SQL statement or a collection of related SQL statements

```
INSERT INTO INVOICE
VALUES (1009, 10016, '18-Jan-2018', 256.99, 20.56, 277.55, 'cred', 0.00, 277.55);
INSERT INTO LINE
VALUES (1009, 1, '89-WRE-Q', 1, 256.99, 256.99);

UPDATE PRODUCT
SET    PROD_QOH = PROD_QOH - 1
WHERE  PROD_CODE = '89-WRE-Q';

UPDATE CUSTOMER
SET    CUST_BALANCE = CUST_BALANCE + 277.55
WHERE  CUST_NUMBER = 10016;

INSERT INTO ACCT_TRANSACTION
VALUES (10007, '18-Jan-18', 10016, 'charge', 277.55);

COMMIT;
```

Evaluating Transaction Results

FIGURE 10.2 TRACING THE TRANSACTION IN THE CH10_SALECO DATABASE

Table name: INVOICE

INV NUMBER	DATE	INV DATE	INV STATUS	INV TYPE	INV TOTAL	INV TAX	INV TAX TYPE	INV TAX AMOUNT	INV BALANCE
1001	10-Jan-18	10-Jan-18	0	0	10.00	0.00		0.00	10.00
1002	10-Jan-18	10-Jan-18	0	0	10.00	0.00		0.00	10.00
1003	10-Jan-18	10-Jan-18	0	0	10.00	0.00		0.00	10.00
1004	10-Jan-18	10-Jan-18	0	0	10.00	0.00		0.00	10.00
1005	10-Jan-18	10-Jan-18	0	0	10.00	0.00		0.00	10.00
1006	10-Jan-18	10-Jan-18	0	0	10.00	0.00		0.00	10.00
1007	10-Jan-18	10-Jan-18	0	0	10.00	0.00		0.00	10.00
1008	10-Jan-18	10-Jan-18	0	0	10.00	0.00		0.00	10.00
1009	10-Jan-18	10-Jan-18	0	0	10.00	0.00		0.00	10.00
1010	10-Jan-18	10-Jan-18	0	0	10.00	0.00		0.00	10.00

Table name: PRODUCT

PROD_CODE	PROD_DESCRIPTION	PROD_QUANTITY	PROD_UNIT	PROD_PRICE	PROD_DISCOUNT	PROD_AMOUNT
100001	Power supply, 150W, 3-phase	10	W	100.00	0.00	1000.00
100002	2.5kW gen. set, 150W, 3-phase	10	W	100.00	0.00	1000.00
100003	100W gen. set, 150W, 3-phase	10	W	100.00	0.00	1000.00
100004	100W gen. set, 150W, 3-phase	10	W	100.00	0.00	1000.00
100005	100W gen. set, 150W, 3-phase	10	W	100.00	0.00	1000.00
100006	100W gen. set, 150W, 3-phase	10	W	100.00	0.00	1000.00
100007	100W gen. set, 150W, 3-phase	10	W	100.00	0.00	1000.00
100008	100W gen. set, 150W, 3-phase	10	W	100.00	0.00	1000.00
100009	100W gen. set, 150W, 3-phase	10	W	100.00	0.00	1000.00
100010	100W gen. set, 150W, 3-phase	10	W	100.00	0.00	1000.00

Table name: CUSTOMER

CUST_CODE	CUST_NAME	CUST_PHONE	CUST_EMAIL	CUST_ADDRESS	CUST_CITY	CUST_STATE	CUST_COUNTRY	CUST_BALANCE
10001	John	1000	1000	1000	1000	1000	1000	1000
10002	John	1000	1000	1000	1000	1000	1000	1000
10003	John	1000	1000	1000	1000	1000	1000	1000
10004	John	1000	1000	1000	1000	1000	1000	1000
10005	John	1000	1000	1000	1000	1000	1000	1000
10006	John	1000	1000	1000	1000	1000	1000	1000
10007	John	1000	1000	1000	1000	1000	1000	1000
10008	John	1000	1000	1000	1000	1000	1000	1000
10009	John	1000	1000	1000	1000	1000	1000	1000
10010	John	1000	1000	1000	1000	1000	1000	1000

Table name: ACCT_TRANSACTION

ACCT_TRANS_NUM	ACCT_TRANS_DATE	ACCT_TRANS_TYPE	ACCT_TRANS_AMOUNT
10001	10-Jan-18	10011 change	10.00
10002	10-Jan-18	10011 change	10.00
10003	10-Jan-18	10011 change	10.00
10004	10-Jan-18	10011 change	10.00
10005	10-Jan-18	10011 change	10.00
10006	10-Jan-18	10011 change	10.00
10007	10-Jan-18	10011 change	10.00
10008	10-Jan-18	10011 change	10.00
10009	10-Jan-18	10011 change	10.00
10010	10-Jan-18	10011 change	10.00

Database name: Ch10_SaleCo

Table name: LINE

LINE_NUMBER	LINE_DATE	LINE_PRICE	LINE_DISCOUNT	LINE_AMOUNT
1001	10-Jan-18	10.00	0.00	10.00
1002	10-Jan-18	10.00	0.00	10.00
1003	10-Jan-18	10.00	0.00	10.00
1004	10-Jan-18	10.00	0.00	10.00
1005	10-Jan-18	10.00	0.00	10.00
1006	10-Jan-18	10.00	0.00	10.00
1007	10-Jan-18	10.00	0.00	10.00
1008	10-Jan-18	10.00	0.00	10.00
1009	10-Jan-18	10.00	0.00	10.00
1010	10-Jan-18	10.00	0.00	10.00

Evaluating Transaction Results

- Not all transactions update database
 - SQL code represents a transaction because it accesses a database
- Improper or incomplete transactions can have devastating effect on database integrity
 - Users can define enforceable constraints based on business rules
 - Other integrity rules are automatically enforced by the DBMS

Transaction Properties

Atomicity

- All operations of a transaction must be completed
- If not, the transaction is aborted

Consistency

- Permanence of database's consistent state

Isolation

- Data used during transaction cannot be used by second transaction until the first is completed

Durability

- Ensures that once transactions are committed, they cannot be undone or lost

Serializability

- Ensures that the schedule for the concurrent execution of several transactions should yield consistent results

Transaction Management with SQL

- SQL statements that provide transaction support
 - COMMIT
 - ROLLBACK
- Transaction sequence must continue until:
 - COMMIT statement is reached
 - ROLLBACK statement is reached
 - End of program is reached
 - Program is abnormally terminated

Transaction Log

- Keeps track of all transactions that update the database
- DBMS uses the information stored in a log for:
 - Recovery requirement triggered by a ROLLBACK statement
 - A program's abnormal termination
 - A system failure

Transaction Log

- The transaction log stores the following:
- A record for the beginning of the transaction.
- For each transaction component (SQL statement):
 - The type of operation being performed (INSERT, UPDATE, DELETE).
 - The names of the objects affected by the transaction (the name of the table).
 - The “before” and “after” values for the fields being updated.
 - Pointers to the previous and next transaction log entries for the same transaction.
- The ending (COMMIT) of the transaction.

Transaction Log

TABLE 10.1

A TRANSACTION LOG

TRL_ID	TRX_NUM	PREV_PTR	NEXT_PTR	OPERATION	TABLE	ROW ID	ATTRIBUTE	BEFORE VALUE	AFTER VALUE
341	101	Null	352	START	****Start Transaction				
352	101	341	363	UPDATE	PRODUCT	1558-QW1	PROD_QOH	25	23
363	101	352	365	UPDATE	CUSTOMER	10011	CUST_BALANCE	525.75	615.73
365	101	363	Null	COMMIT	**** End of Transaction				



TRL_ID = Transaction log record ID

TRX_NUM = Transaction number

PTR = Pointer to a transaction log record ID

(Note: The transaction number is automatically assigned by the DBMS.)

Concurrency Control

- Coordination of the simultaneous transactions execution in a multiuser database system
- Objective - Ensures serializability of transactions in a multiuser database environment

Problems in Concurrency Control

Lost update

- Occurs in two concurrent transactions when:
 - Same data element is updated
 - One of the updates is lost

Uncommitted data

- Occurs when:
 - Two transactions are executed concurrently
 - First transaction is rolled back after the second transaction has already accessed uncommitted data

Inconsistent retrievals

- Occurs when a transaction accesses data before and after one or more other transactions finish working with such data

Lost update

TABLE 10.2

TWO CONCURRENT TRANSACTIONS TO UPDATE QOH

TRANSACTION	COMPUTATION
T1: Purchase 100 units	$PROD_QOH = PROD_QOH + 100$
T2: Sell 30 units	$PROD_QOH = PROD_QOH - 30$

Table 10.3 shows the serial execution of the transactions under normal circumstances, yielding the correct answer $PROD_QOH = 105$.

TABLE 10.3

SERIAL EXECUTION OF TWO TRANSACTIONS

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read $PROD_QOH$	35
2	T1	$PROD_QOH = 35 + 100$	
3	T1	Write $PROD_QOH$	135
4	T2	Read $PROD_QOH$	135
5	T2	$PROD_QOH = 135 - 30$	
6	T2	Write $PROD_QOH$	105

Lost update

- However, suppose that a transaction can read a product's PROD_QOH value from the table *before* a previous transaction has been committed, using the *same* product.
- The sequence depicted in Table 10.4 shows how the lost update problem can arise.
- Note that the first transaction (T1) has not yet been committed when the second transaction (T2) is executed.
- Therefore, T2 still operates on the value 35, and its subtraction yields 5 in memory.
- In the meantime, T1 writes the value 135 to disk, which is promptly overwritten by T2. In short, the addition of 100 units is “lost” during the process.

Lost update

TABLE 10.4

LOST UPDATES

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T2	Read PROD_QOH	35
3	T1	$PROD_QOH = 35 + 100$	
4	T2	$PROD_QOH = 35 - 30$	
5	T1	Write PROD_QOH (lost update)	135
6	T2	Write PROD_QOH	5

Uncommitted data

TABLE 10.5

TRANSACTIONS CREATING AN UNCOMMITTED DATA PROBLEM

TRANSACTION	COMPUTATION
T1: Purchase 100 units	$PROD_QOH = PROD_QOH + 100$ (Rolled back)
T2: Sell 30 units	$PROD_QOH = PROD_QOH - 30$

Table 10.6 shows how the serial execution of these transactions yields the correct answer under normal circumstances.

TABLE 10.6

CORRECT EXECUTION OF TWO TRANSACTIONS

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	$PROD_QOH = 35 + 100$	
3	T1	Write PROD_QOH	135
4	T1	*****ROLLBACK *****	35
5	T2	Read PROD_QOH	35
6	T2	$PROD_QOH = 35 - 30$	
7	T2	Write PROD_QOH	5

Uncommitted data

Table 10.7 shows how the uncommitted data problem can arise when the ROLLBACK is completed after T2 has begun its execution.

TABLE 10.7

AN UNCOMMITTED DATA PROBLEM

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	$PROD_QOH = 35 + 100$	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH (Read uncommitted data)	135
5	T2	$PROD_QOH = 135 - 30$	
6	T1	***** ROLLBACK *****	35
7	T2	Write PROD_QOH	105

Inconsistent retrievals

TABLE 10.8

RETRIEVAL DURING UPDATE

TRANSACTION T1	TRANSACTION T2
SELECT SUM(PROD_QOH) FROM PRODUCT	UPDATE PRODUCT SET PROD_QOH = PROD_QOH + 10 WHERE PROD_CODE = 1546-QQ2
	UPDATE PRODUCT SET PROD_QOH = PROD_QOH - 10 WHERE PROD_CODE = 1558-QW1
	COMMIT;

- While T1 calculates the total quantity on hand (PROD_QOH) for all items, T2 represents the correction of a typing error: the user added 10 units to product 1558-QW1's PROD_QOH but meant to add the 10 units to product 1546-QQ2's PROD_QOH.
- To correct the problem, the user adds 10 to product 1546-QQ2's PROD_QOH and subtracts 10 from product 1558-QW1's PROD_QOH.
- The initial and final PROD_QOH values are reflected in Table 10.9. (Only a few PROD_CODE values are shown for the PRODUCT table. To illustrate the point, the sum for the PROD_QOH values is shown for these few products.)

Inconsistent retrievals

TABLE 10.9

TRANSACTION RESULTS: DATA ENTRY CORRECTION

	BEFORE	AFTER
PROD_CODE	PROD_QOH	PROD_QOH
11QER/31	8	8
13-Q2/P2	32	32
1546-QQ2	15	$(15 + 10) \rightarrow 25$
1558-QW1	23	$(23 - 10) \rightarrow 13$
2232-QTY	8	8
2232-QWE	6	6
Total	92	92

Inconsistent retrievals

TABLE 10.10

INCONSISTENT RETRIEVALS

TIME	TRANSACTION	ACTION	VALUE	TOTAL
1	T1	Read PROD_QOH for PROD_CODE = '11QER/31'	8	8
2	T1	Read PROD_QOH for PROD_CODE = '13-Q2/P2'	32	40
3	T2	Read PROD_QOH for PROD_CODE = '1546-QQ2'	15	
4	T2	PROD_QOH = 15 + 10		
5	T2	Write PROD_QOH for PROD_CODE = '1546-QQ2'	25	
6	T1	Read PROD_QOH for PROD_CODE = '1546-QQ2'	25	(After) 65
7	T1	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	(Before) 88
8	T2	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	
9	T2	PROD_QOH = 23 - 10		
10	T2	Write PROD_QOH for PROD_CODE = '1558-QW1'	13	
11	T2	***** COMMIT *****		
12	T1	Read PROD_QOH for PROD_CODE = '2232-QTY'	8	96
13	T1	Read PROD_QOH for PROD_CODE = '2232-QWE'	6	102

The Scheduler

- Establishes the order in which the operations are executed within concurrent transactions
 - Interleaves the execution of database operations to ensure serializability and isolation of transactions
- Based on concurrent control algorithms to determine the appropriate order
- Creates serialization schedule
 - **Serializable schedule:** Interleaved execution of transactions yields the same results as the serial execution of the transactions

The Scheduler

TABLE 10.11

READ/WRITE CONFLICT SCENARIOS: CONFLICTING DATABASE OPERATIONS MATRIX

	TRANSACTIONS		RESULT
	T1	T2	
Operations	Read	Read	No conflict
	Read	Write	Conflict
	Write	Read	Conflict
	Write	Write	Conflict

Concurrency Control with Locking Methods

- Locking methods - Facilitate isolation of data items used in concurrently executing transactions
- **Lock:** Guarantees exclusive use of a data item to a current transaction
- **Pessimistic locking:** Use of locks based on the assumption that conflict between transactions is likely
- **Lock manager:** Responsible for assigning and policing the locks used by the transactions

Lock Granularity

- Indicates the level of lock use
- Levels of locking
 - **Database-level lock**
 - **Table-level lock**
 - **Page-level lock**
 - **Page or diskpage:** Directly addressable section of a disk
 - **Row-level lock**
 - **Field-level lock**

Figure 10.3 - Database-Level Locking Sequence

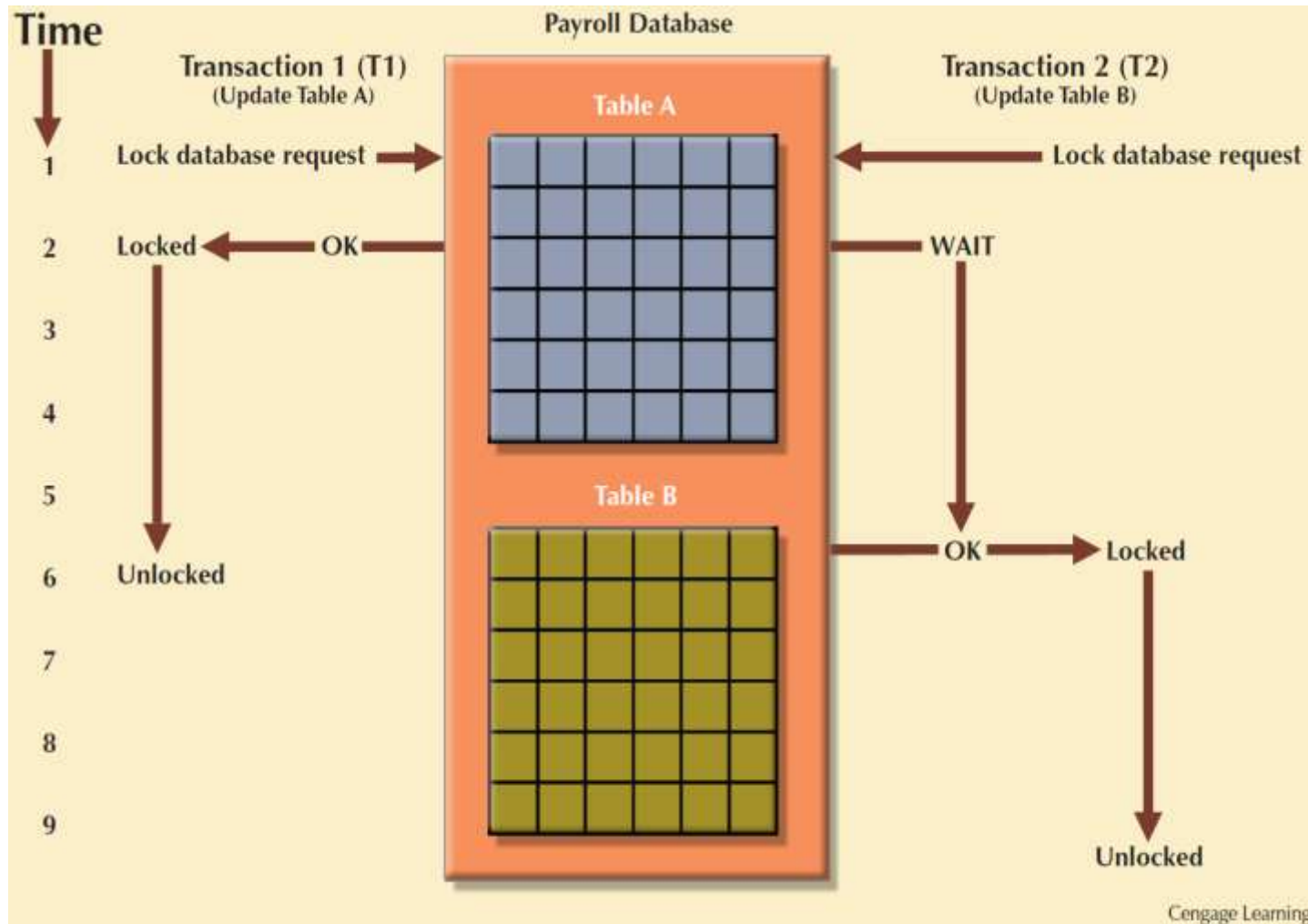


Figure 10.4 - An Example of a Table-Level Lock

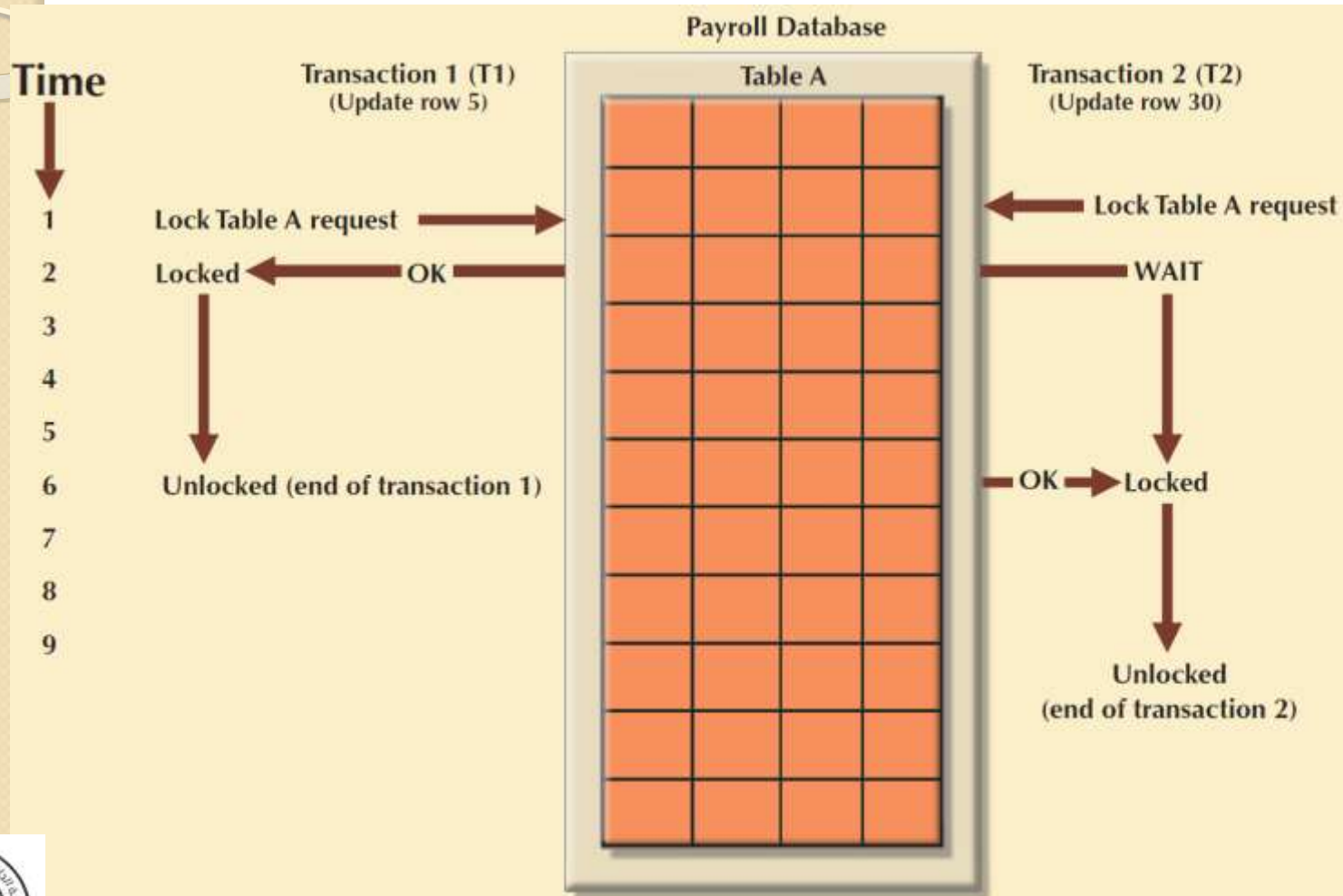
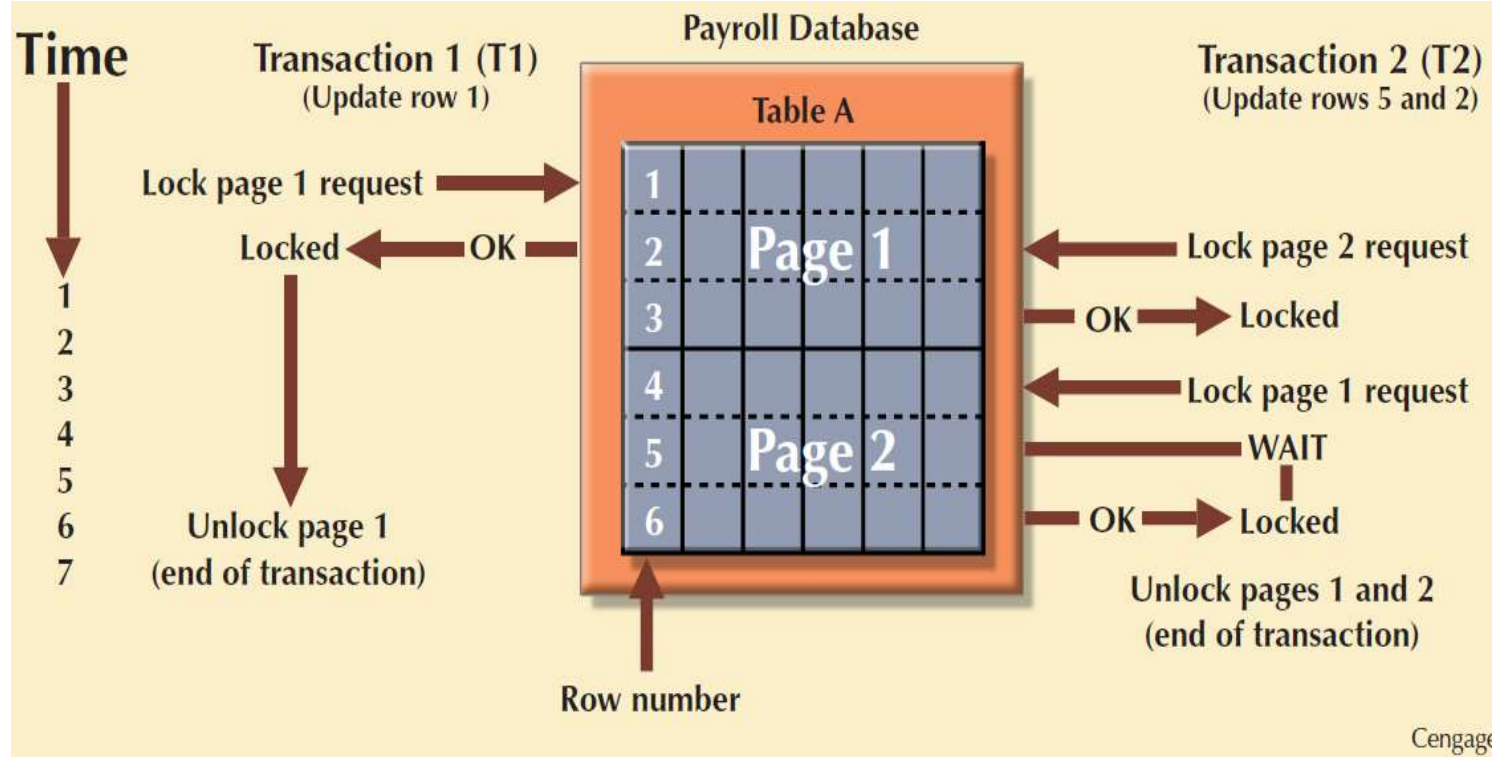
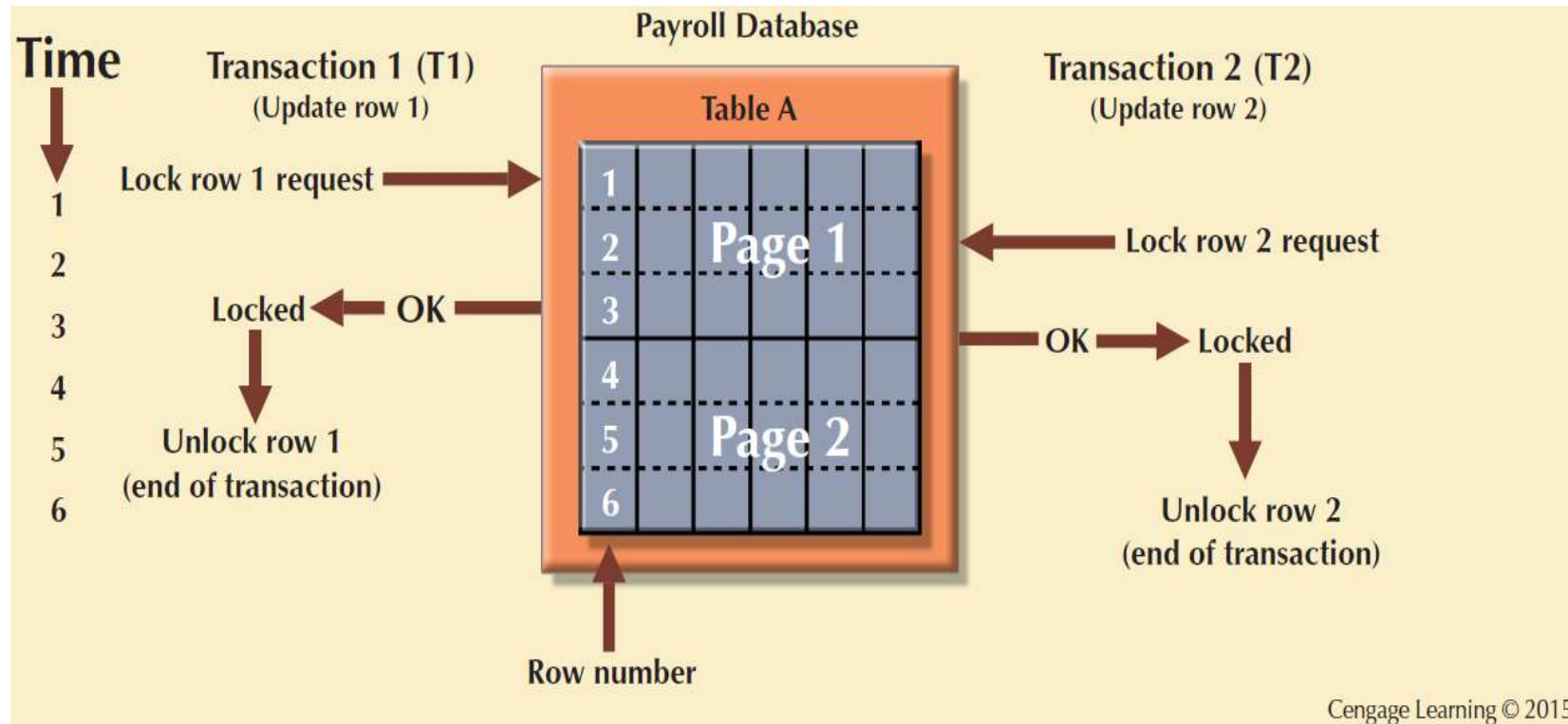


Figure 10.5 - An Example of a Page-Level Lock



Cengage Learning © 2015

Figure 10.6 - An Example of a Row-Level Lock



Lock Types

Binary lock

- Has two states, locked (1) and unlocked (0)
- If an object is locked by a transaction, no other transaction can use that object
- If an object is unlocked, any transaction can lock the object for its use

Exclusive lock

- Exists when access is reserved for the transaction that locked the object

Shared lock

- Exists when concurrent transactions are granted read access on the basis of a common lock

Problems in Using Locks

- Resulting transaction schedule might not be serializable
- Schedule might create **deadlocks**

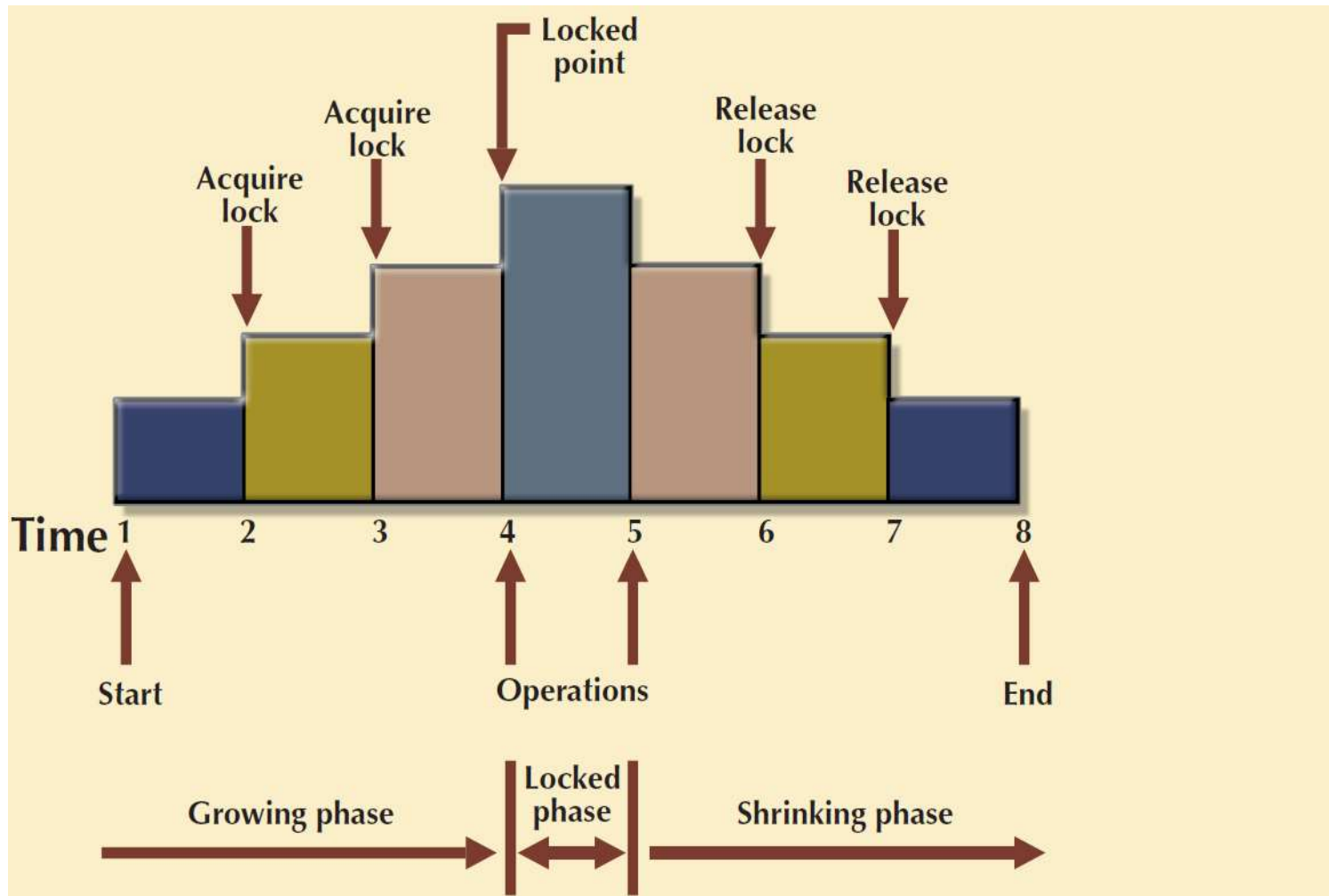
Two-Phase Locking (2PL)

- Defines how transactions acquire and relinquish locks
- Guarantees serializability but does not prevent deadlocks
- Phases
 - Growing phase - Transaction acquires all required locks without unlocking any data
 - Shrinking phase - Transaction releases all locks and cannot obtain any new lock

Two-Phase Locking (2PL)

- Governing rules
 - Two transactions cannot have conflicting locks
 - No unlock operation can precede a lock operation in the same transaction
 - No data are affected until all locks are obtained

Figure 10.7 - Two-Phase Locking Protocol



Deadlocks

- Occurs when two transactions wait indefinitely for each other to unlock data
 - Known as **deadly embrace**
- Control techniques
 - Deadlock prevention
 - Deadlock detection
 - Deadlock avoidance
- Choice of deadlock control method depends on database environment

Table 10.13 - How a Deadlock Condition is Created

TIME	TRANSACTION	REPLY	LOCK STATUS	
0			Data X	Data Y
1	T1:LOCK(X)	OK	Unlocked	Unlocked
2	T2: LOCK(Y)	OK	Locked	Unlocked
3	T1:LOCK(Y)	WAIT	Locked	Locked
4	T2:LOCK(X)	WAIT	Locked	Locked
5	T1:LOCK(Y)	WAIT	Locked	Locked
6	T2:LOCK(X)	WAIT	Locked	Locked
7	T1:LOCK(Y)	WAIT	Locked	Locked
8	T2:LOCK(X)	WAIT	Locked	Locked
9	T1:LOCK(Y)	WAIT	Locked	Locked
...				
...				
...				
...				



Cengage Learning © 2015

Time Stamping

- Assigns global, unique time stamp to each transaction
 - Produces explicit order in which transactions are submitted to DBMS
- Properties
 - **Uniqueness:** Ensures no equal time stamp values exist
 - **Monotonicity:** Ensures time stamp values always increases

Time Stamping

- Disadvantages
 - Each value stored in the database requires two additional stamp fields
 - Increases memory needs
 - Increases the database's processing overhead
 - Demands a lot of system resources

Table 10.14 - Wait/Die and Wound/Wait Concurrency Control Schemes

TRANSACTION REQUESTING LOCK	TRANSACTION OWNING LOCK	WAIT/DIE SCHEME	WOUND/WAIT SCHEME
T1 (11548789)	T2 (19562545)	T1 waits until T2 is completed and T2 releases its locks.	- T1 preempts (rolls back) T2. - T2 is rescheduled using the same timestamp.
T2 (19562545)	T1 (11548789)	- T2 dies (rolls back). - T2 is rescheduled using the same times tamp.	T2 waits until T1 is completed and T1 releases its locks.

Cengage Learning © 2015

Concurrency Control with Optimistic Methods

- **Optimistic approach:** Based on the assumption that the majority of database operations do not conflict
 - Does not require locking or time stamping techniques
 - Transaction is executed without restrictions until it is committed

Phases of Optimistic Approach

Read

- Transaction:
 - Reads the database
 - Executes the needed computations
 - Makes the updates to a private copy of the database values

Validation

- Transaction is validated to ensure that the changes made will not affect the integrity and consistency of the database

Write

- Changes are permanently applied to the database

Table 10.15 - Transaction Isolation Levels

	Isolation Level	Allowed			Comment
		Dirty Read	Nonrepeatable Read	Phantom Read	
Less restrictive ↑ ↓ More restrictive	Read Uncommitted	Y	Y	Y	The transaction reads uncommitted data, allows nonrepeatable reads and, phantom reads.
	Read Committed	N	Y	Y	Does not allow uncommitted data reads but allows nonrepeatable reads and phantom reads.
	Repeatable Read	N	N	Y	Only allows phantom reads.
	Serializable	N	N	N	Does not allow dirty reads, nonrepeatable reads, or phantom reads.
Oracle / SQL Server Only	Read Only / Snapshot	N	N	N	Supported by Oracle and SQL Server. The transaction can only see the changes that were committed at the time the transaction started.

Cengage Learning © 2015

Database Recovery Management

- **Database recovery:** Restores database from a given state to a previously consistent state
- Recovery transactions are based on the atomic transaction property
 - **Atomic transaction property:** All portions of a transaction must be treated as a single logical unit of work
 - If transaction operation cannot be completed:
 - Transaction must be aborted
 - Changes to database must be rolled back

Concepts that Affect Transaction Recovery

Deferred-write technique or deferred update

- Ensures that transaction logs are always written before the data are updated

Redundant transaction logs

- Ensure that a physical disk failure will not impair the DBMS's ability to recover data

Buffers

- Temporary storage areas in a primary memory

Checkpoints

- Allows DBMS to write all its updated buffers in memory to disk

Techniques used in Transaction Recovery Procedures

Deferred-write technique or deferred update

- Only transaction log is updated

Write-through technique or immediate update

- Database is immediately updated by transaction operations during transaction's execution

Recovery Process in Deferred-Write Technique

- Identify the last check point in the transaction log
- If transaction was committed before the last check point
 - Nothing needs to be done
- If transaction was committed after the last check point
 - Transaction log is used to redo the transaction
- If transaction had a ROLLBACK operation after the last check point
 - Nothing needs to be done

Recovery Process in Write-Through Technique

- Identify the last checkpoint in the transaction log
- If transaction was committed before the last checkpoint
 - Nothing needs to be done
- If transaction was committed after the last checkpoint
 - Transaction must be redone
- If transaction had a ROLLBACK operation after the last checkpoint
 - Transaction log is used to ROLLBACK the operations

Database systems ITCS2322

Chapter II : Database Performance Tuning and Query Optimization

اعداد : دكتور محمد عبد اللطيف راضي

Learning Objectives

- In this chapter, the student will learn:
 - Basic database performance-tuning concepts
 - How a DBMS processes SQL queries
 - About the importance of indexes in query processing
 - About the types of decisions the query optimizer has to make
 - Some common practices used to write efficient SQL code
 - How to formulate queries and tune the DBMS for optimal performance

Database Performance-Tuning Concepts

- End users interact with the DBMS through the use of queries to generate information, using the following sequence:
 1. The end-user (client-end) application generates a query.
 2. The query is sent to the DBMS (server end).
 3. The DBMS (server end) executes the query.
 4. The DBMS sends the resulting data set to the end-user (client-end) application.
- Goal of database performance is to execute queries as fast as possible

Database Performance-Tuning Concepts

- **Database performance tuning:** Set of activities and procedures that reduce response time of database system.
- The time required by a query to return a result set depends on many factors.
 - CPU,
 - (RAM), and
 - input/output (hard disk and network)

Table 11.1 - General Guidelines for Better System Performance

TABLE 11.1

GENERAL GUIDELINES FOR BETTER SYSTEM PERFORMANCE

	SYSTEM RESOURCES	CLIENT	SERVER
Hardware	CPU	The fastest possible Dual-core CPU or higher "Virtualized Client desktop technologies could also be used."	The fastest possible Multiple processors (quad-core technology or higher) Cluster of networked computers "Virtualized server technology could be used"
	RAM	The maximum possible to avoid OS memory to disk swapping	The maximum possible to avoid OS memory to disk swapping
	Storage	Fast SATA/EIDE hard disk with sufficient free hard disk space Solid state drives (SSDs) for faster speed	Multiple high-speed, high-capacity disks Fast disk interface (SAS / SCSI / Firewire / Fibre Channel) RAID configuration optimized for throughput Solid state drives (SSDs) for faster speed Separate disks for OS, DBMS, and data spaces
	Network	High-speed connection	High-speed connection
Software	Operating system (OS)	64-bit OS for larger address spaces Fine-tuned for best client application performance	64-bit OS for larger address spaces Fine-tuned for best server application performance
	Network	Fine-tuned for best throughput	Fine-tuned for best throughput
	Application	Optimize SQL in client application	Optimize DBMS server for best performance

Database Performance-Tuning Concepts

- Fine-tuning the performance of a system requires that all factors must operate at optimum level with minimal bottlenecks.
- **Good database performance starts with good database design.**
- *No amount of fine tuning will make a poorly designed database perform as well as a well-designed database*

Performance Tuning: Client and Server

- Client side
 - **SQL performance tuning:** Generates SQL query that returns correct answer in least amount of time
 - Using minimum amount of resources at server
- Server side
 - **DBMS performance tuning:** DBMS environment configured to respond to clients' requests as fast as possible
 - Optimum use of existing resources

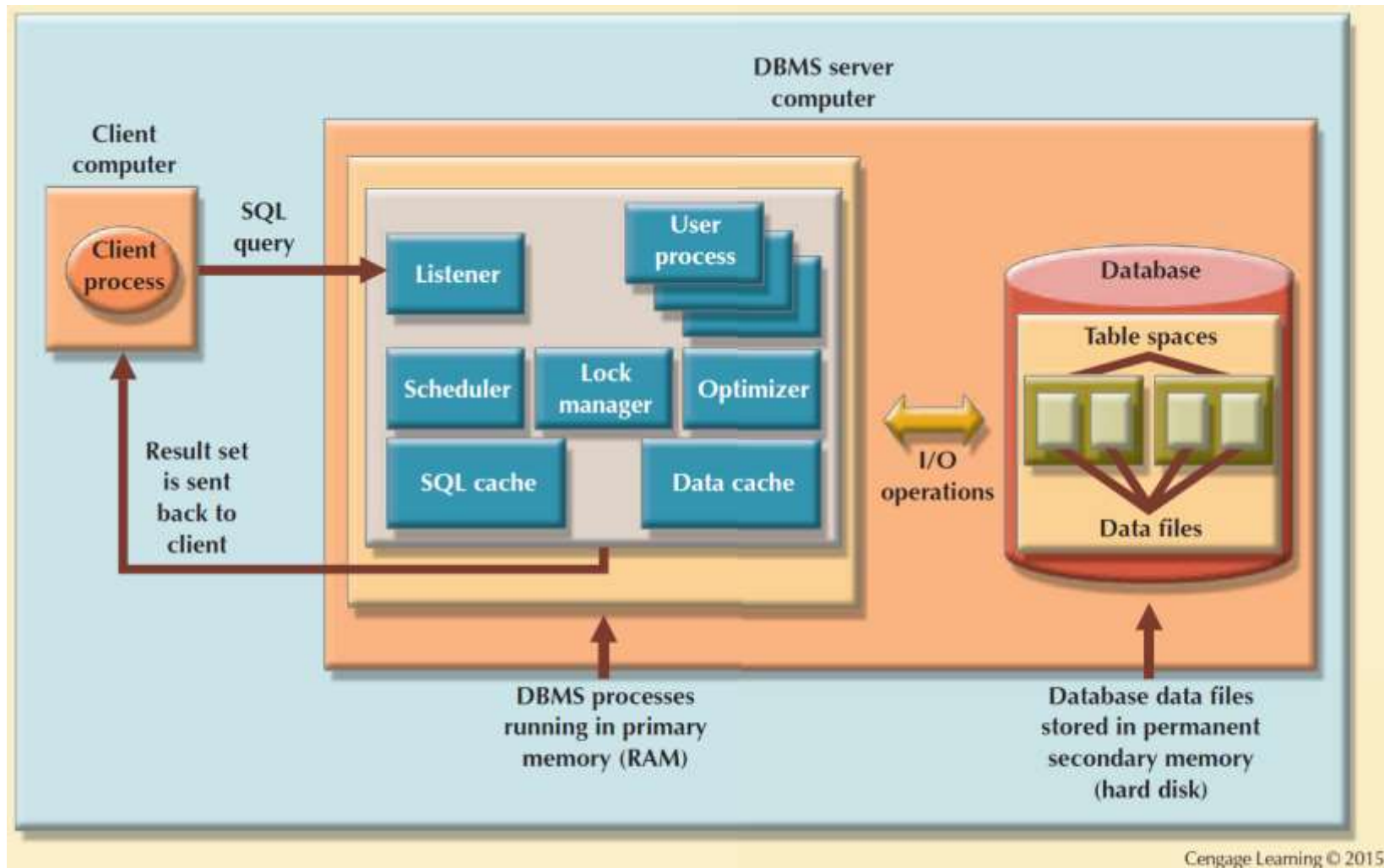
DBMS Architecture

- All data in a database are stored in **data files**
 - Data files automatically expand in predefined increments known as **extends**
- Data files are grouped in file groups or table spaces
 - **Table space** or **file group**: Logical grouping of several data files that store data with similar characteristics
- **Data cache** or **buffer cache**: Shared, reserved memory area
 - Stores most recently accessed data blocks in RAM

DBMS Architecture

- **SQL cache or procedure cache:** Stores most recently executed SQL statements or PL/SQL procedures
- DBMS retrieves data from permanent storage and places them in RAM
- **Input/output request:** Low-level data access operation that reads or writes data to and from computer devices
- Data cache is faster than working with data files
- Majority of performance-tuning activities focus on minimizing I/O operations

Figure 11.1 - Basic DBMS Architecture



Database Query Optimization Modes

- Algorithms proposed for query optimization are based on:
 1. Selection of the optimum order to achieve the fastest execution time
 2. Selection of sites to be accessed to minimize communication costs
- Evaluated on the basis of:
 - Operation mode
 - Timing of its optimization

Classification of Operation Modes

- **Automatic query optimization:**
DBMS finds the most cost-effective access path without user intervention
- **Manual query optimization:** Requires that the optimization be selected and scheduled by the end user or programmer

Classification Based on Timing of Optimization

- **Static query optimization:** best optimization strategy is selected when the query is compiled by the DBMS
 - Takes place at compilation time
- **Dynamic query optimization:** Access strategy is dynamically determined by the DBMS at run time, using the most up-to-date information about the database
 - Takes place at execution time

Classification Based on Type of Information Used to Optimize the Query

- **Statistically based query optimization algorithm:** Statistics are used by the DBMS to determine the best access strategy
- Statistical information is generated by DBMS through:
 - **Dynamic statistical generation mode**
 - **Manual statistical generation mode**
- **Rule-based query optimization algorithm:** based on a set of user-defined rules to determine the best query access strategy

Table 11.2 - Sample Database Statistics Measurements

TABLE 11.2

SAMPLE DATABASE STATISTICS MEASUREMENTS

DATABASE OBJECT	SAMPLE MEASUREMENTS
Tables	Number of rows, number of disk blocks used, row length, number of columns in each row, number of distinct values in each column, maximum value in each column, minimum value in each column, and columns that have indexes
Indexes	Number and name of columns in the index key, number of key values in the index, number of distinct key values in the index key, histogram of key values in an index, and number of disk pages used by the index
Environment Resources	Logical and physical disk block size, location and size of data files, and number of extents per data file

Query Processing

Parsing

- DBMS parses the SQL query and chooses the most efficient access/execution plan

Execution

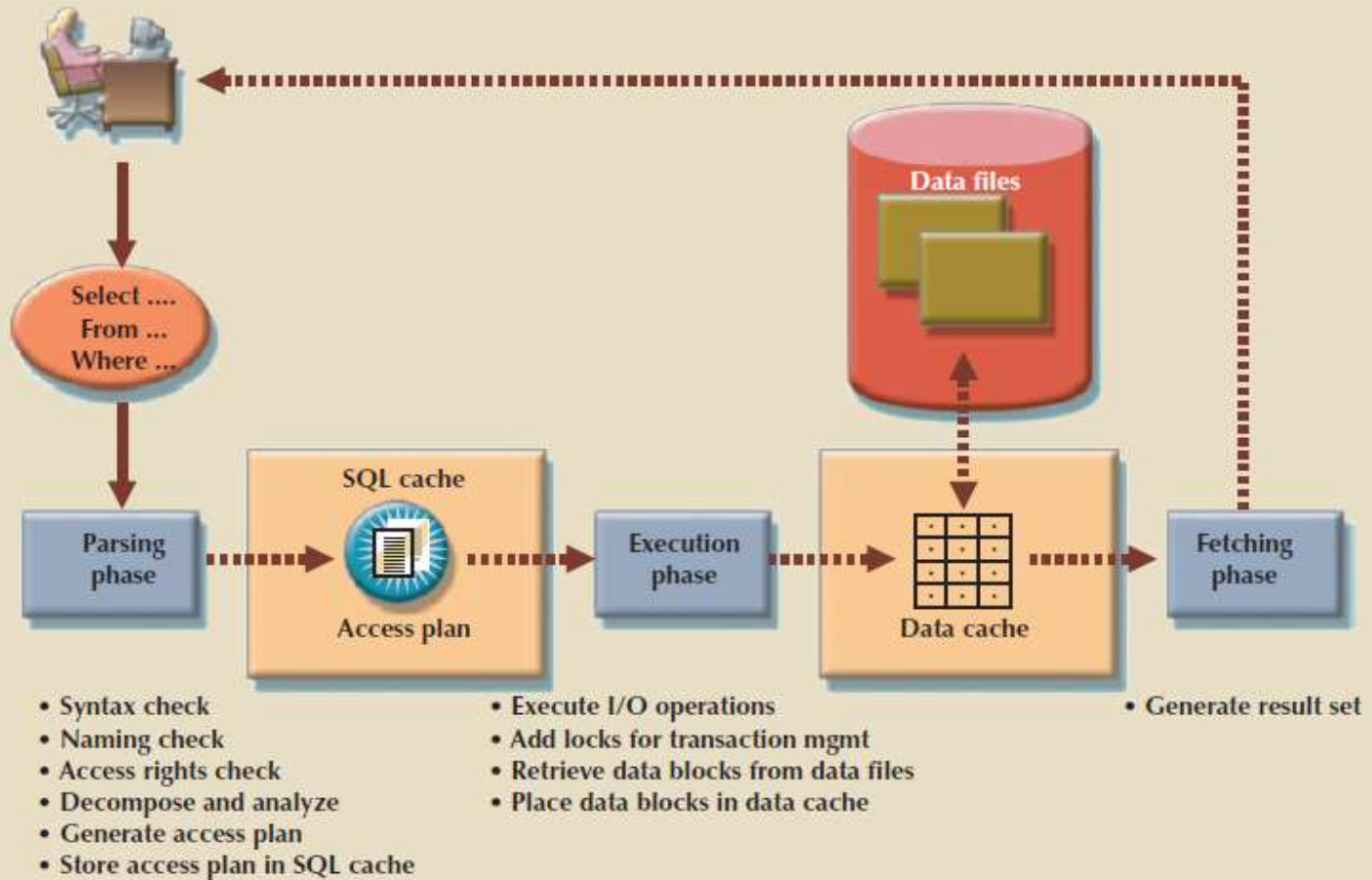
- DBMS executes the SQL query using the chosen execution plan

Fetching

- DBMS fetches the data and sends the result set back to the client

Query Processing

FIGURE 11.2 QUERY PROCESSING



SQL Parsing Phase

- Query is broken down into smaller units
- Original SQL query is transformed into slightly different version of the original SQL code which is fully equivalent and more efficient
- **Query optimizer:** Analyzes SQL query and finds most efficient way to access data
- **Access plans:** DBMS-specific and translate client's SQL query into a series of complex I/O operations

SQL Parsing Phase

- If access plan already exists for query in SQL cache, DBMS reuses it
 - If not, optimizer evaluates various plans and chooses one to be placed in SQL cache for use

SQL Parsing Phase

- Parsing a SQL query requires several steps:
 1. Validated for syntax compliance
 2. Validated against the data dictionary to ensure that table names and column names are correct.
 3. Validated against the data dictionary to ensure that the user has proper access rights
 4. Analyzed and decomposed into more atomic components
 5. Optimized through transformation into a fully equivalent but more efficient SQL query
 6. Prepared for execution by determining the most efficient execution or access plan

SQL Execution Phase

- All I/O operations indicated in the access plan are executed
 - Locks are acquired
 - Data are retrieved and placed in data cache
 - Transaction management commands are processed

SQL Fetching Phase

- Rows of resulting query result set are returned to client
 - DBMS may use temporary table space to store temporary data
 - Database server coordinates the movement of the result set rows from the server cache to the client cache

Query Processing Bottlenecks

- Delay introduced in the processing of an I/O operation that slows the system
- Caused by the:
 - CPU
 - RAM
 - Hard disk
 - Network
 - Application code

Indexes and Query Optimization

- Indexes
 - Help speed up data access
 - Facilitate searching, sorting, using aggregate functions, and join operations
 - Ordered set of values that contain the index key and pointers
 - More efficient than a full table scan

Indexes and Query Optimization

FIGURE 11.3 INDEX REPRESENTATION FOR THE CUSTOMER TABLE

CUSTOMER TABLE
(14,786 rows)

STATE_NDX INDEX

Key	Row	RowID	CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE	CUS_STATE	CUS_BALANCE
AZ	2	1	10010	Ramas	Alfred	A	615	944-2573	FL	90.00
		2	10011	Durne	Leona	K	713	994-1289	AZ	90.00
		3	10012	Smith	Kathy	W	615	994-7385	TX	\$345.86
FL	1	4	10013	Olewski	Paul	F	615	994-2190	AZ	\$536.75
FL	7	5	10014	Orlando	Myron		615	223-1672	NY	90.00
FL	8	6	10015	O'Brian	Amy	B	713	443-3381	NY	90.00
FL	13245	7	10016	Brown	James	G	615	297-1228	FL	\$221.19
FL	14706	8	10017	Williams	George		615	290-2555	FL	\$768.93
		9	10018	Farriss	Anne	G	713	302-7185	TX	\$216.55
		10	10019	Smith	Olette	K	615	297-3839	AZ	90.00
		1000								
		1000								
		13245	23120	Vernh	George	D	415	231-9872	FL	\$675.00
		1000								
		14706	24560	Suzrat	Victor		435	342-9076	FL	\$342.00

Indexes and Query Optimization

- One measure that determines the need for an index is the data *sparsity* of the column you want to index.
- **Data sparsity:** Number of different values a column could have.
- Knowing the sparsity helps you decide whether the use of an index is appropriate
 - column with high sparsity: index processing might be necessary work
 - column with low sparsity: index processing might be unnecessary work.

Indexes and Query Optimization

- Data structures used to implement indexes:
 - **Hash indexes**
 - **B-tree indexes**
 - **Bitmap indexes**
- DBMSs determine best type of index to use

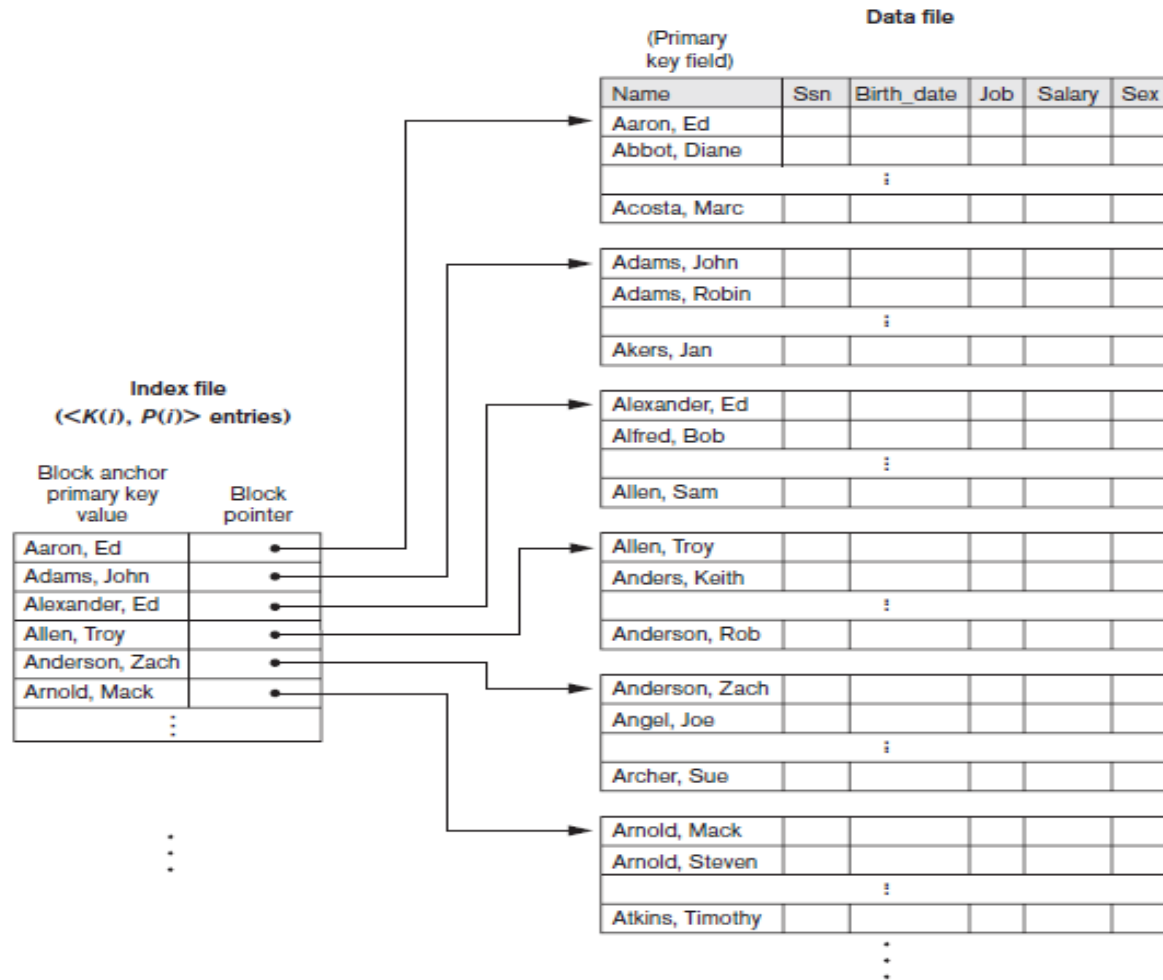
Indexes and Query Optimization

- **Hash index.** A hash index is based on an ordered list of hash values.
- A hash algorithm is used to create a hash value from a key column.
- This value points to an entry in a hash table, which in turn points to the actual location of the data row.
- **This type of index is good for simple and fast lookup operations based on equality conditions.**
- For example, LNAME="Scott" and FNAME="Shannon".

Primary Indexes

- Ordered file with two fields
 - Primary key, $K(i)$
 - Pointer to a disk block, $P(i)$
- One index entry in the index file for each block in the data file
- Indexes may be dense or sparse
 - Dense index has an index entry for every search key value in the data file
 - Sparse index has entries for only some search values

Primary Indexes



Primary Indexes

- Major problem: insertion and deletion of records
 - Move records around and change index values
 - Solutions
 - Use unordered overflow file
 - Use linked list of overflow records

Indexes and Query Optimization

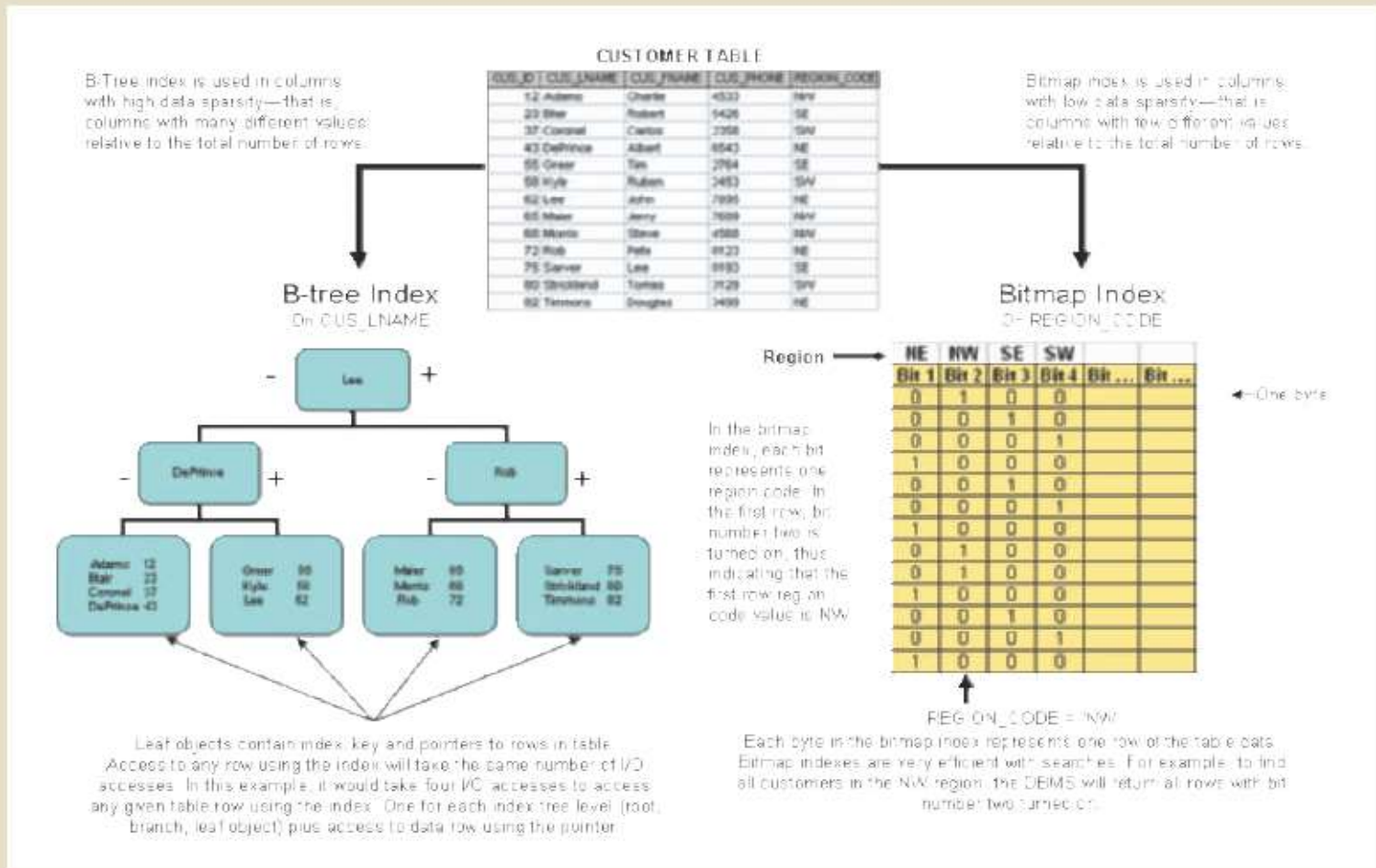
- **The B-tree index** is an ordered data structure organized as an up sided own tree.
- The index tree is stored separately from the data.
- The lower-level leaves of the B-tree index contain the pointers to the actual data rows.
- B-tree indexes are “self-balanced,” which means that it takes approximately the same amount of time to access any given row in the index.
- This is the default and most common type of index used in databases.
- **The B-tree index is used mainly in tables in which column values repeat a relatively small number of times.**

Indexes and Query Optimization

- A **bitmap index** uses a bit array (0s and 1s) to represent the existence of a value or condition.
- These indexes are used mostly in data warehouse applications in tables with a large number of rows in which a small number of column values repeat many times.
- Bitmap indexes tend to use less space than B-tree indexes because they use bits instead of bytes to store their data.

Indexes and Query Optimization

FIGURE 11.4 B-TREE AND BITMAP INDEX REPRESENTATION



Optimizer Choices

- DBMS must choose what indexes to use, how to perform join operations, which table to use first, and so on.
- The query optimizer can operate in one of two modes:
 - **Rule-based optimizer:** Uses preset rules and points to determine the best approach to execute a query
 - **Cost-based optimizer:** Uses algorithms based on statistics about objects being accessed to determine the best approach to execute a query

Using Hints to Affect Optimizer Choices

- Optimizer might not choose the best execution plan
 - Makes decisions based on existing statistics, which might be old
 - Might choose less-efficient decisions
- **Optimizer hints:** Special instructions for the optimizer, embedded in the SQL command text

Table 11.5 - Optimizer Hints

TABLE 11.5

OPTIMIZER HINTS

HINT	USAGE
ALL_ROWS	Instructs the optimizer to minimize the overall execution time—that is, to minimize the time needed to return all rows in the query result set. This hint is generally used for batch mode processes. For example: <pre>SELECT /*+ ALL_ROWS */ * FROM PRODUCT WHERE P_QOH < 10;</pre>
FIRST_ROWS	Instructs the optimizer to minimize the time needed to process the first set of rows—that is, to minimize the time needed to return only the first set of rows in the query result set. This hint is generally used for interactive mode processes. For example: <pre>SELECT /*+ FIRST_ROWS */ * FROM PRODUCT WHERE P_QOH < 10;</pre>
INDEX(name)	Forces the optimizer to use the P_QOH_NDX index to process this query. For example: <pre>SELECT /*+ INDEX(P_QOH_NDX) */ * FROM PRODUCT WHERE P_QOH < 10</pre>

SQL Performance Tuning

- Evaluated from client perspective
 - Most current relational DBMSs perform automatic query optimization at the server end
 - Most SQL performance optimization techniques are DBMS-specific and thus rarely portable
- Majority of performance problems are related to poorly written SQL code

Index Selectivity

- Measure of the likelihood that an index will be used in query processing
- Indexes are used when a subset of rows from a large table is to be selected based on a given condition
- Index cannot always be used to improve performance
- **Function-based index:** Based on a specific SQL function or expression

Conditional Expressions

- Expressed within WHERE or HAVING clauses of a SQL statement
 - Restricts the output of a query to only rows matching conditional criteria
- Guidelines to write efficient conditional expressions in SQL code
 - Use simple columns or literals as operands
 - Numeric field comparisons are faster than character, date, and NULL comparisons

Conditional Expressions

- Equality comparisons are faster than inequality comparisons
- Transform conditional expressions to use literals
- Write equality conditions first when using multiple conditional expressions
- When using multiple AND conditions, write the condition most likely to be false first
- When using multiple OR conditions, put the condition most likely to be true first
- Avoid the use of NOT logical operator

Query Formulation

- Identify what columns and computations are required
- Identify source tables
- Determine how to join tables
- Determine what selection criteria are needed
- Determine the order in which to display the output

DBMS Performance Tuning

- Managing DBMS processes in primary memory and the structures in physical storage
- DBMS performance tuning at server end focuses on setting parameters used for:
 - Data cache
 - SQL cache
 - Sort cache
 - Optimizer mode
- **In-memory database:** Store large portions of the database in primary storage

DBMS Performance Tuning

- Recommendations for physical storage of databases:
 - Use **RAID** (Redundant Array of Independent Disks) to provide a balance between performance improvement and fault tolerance
 - Minimize disk contention
 - Put high-usage tables in their own table spaces
 - Assign separate data files in separate storage volumes for indexes, system, and high-usage tables

DBMS Performance Tuning

- Take advantage of the various table storage organizations in the database
 - **Index-organized table** or **clustered index table**: Stores the end-user data and the index data in consecutive locations in permanent storage
- Partition tables based on usage
- Use denormalized tables where appropriate
- Store computed and aggregate attributes in tables

Database systems ITCS2322

Chapter 12 : Distributed Database Management Systems

اعداد : دكتور محمد عبد اللطيف راضي

Learning Objectives

- In this chapter, the student will learn:
 - About distributed database management systems (DDBMSs) and their components
 - How database implementation is affected by different levels of data and process distribution
 - How transactions are managed in a distributed database environment

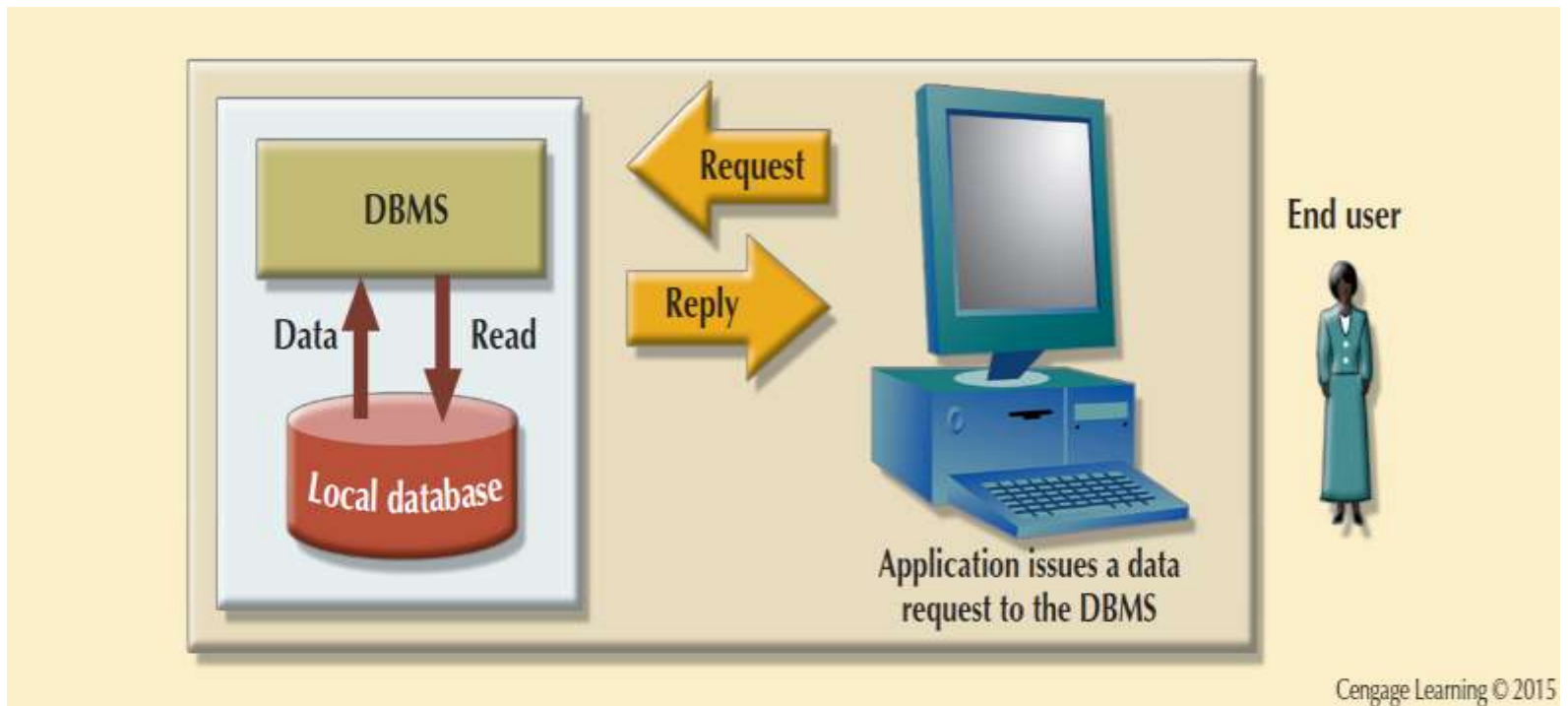
Learning Objectives

- In this chapter, the student will learn:
 - How distributed database design draws on data partitioning and replication to balance performance, scalability, and availability
 - About the trade-offs of implementing a distributed data system

Evolution Database Management Systems

- **Distributed database management system (DDBMS):** Governs storage and processing of logically related data over interconnected computer systems
 - Data and processing functions are distributed among several sites
- **Centralized database management system**
 - Required that corporate data be stored in a single central site
 - Data access provided through dumb terminals

Figure 12.1 - Centralized Database Management System



Factors Affecting the Centralized Database Systems

- Globalization of business operation
- Advancement of web-based services
- Rapid growth of social and network technologies
- Digitization resulting in multiple types of data
- Innovative business intelligence through analysis of data

Factors Affecting the Centralized Database Systems

- Two database requirements became obvious:
 - *Rapid ad hoc data access* became crucial in the quick-response decision-making environment.
 - *Distributed data access* was needed to support geographically dispersed business units.

Factors That Aided DDBMS to Cope With Technological Advancement

Acceptance of Internet as a platform for business

Mobile wireless revolution

Usage of application as a service

Focus on mobile business intelligence

Desirability of Distributed DBMS Over Centralized DBMS

Performance
degradation

High costs

Reliability
problems

Scalability
problems

Organizational
rigidity

Advantages and Disadvantages of DDBMS

Advantages

- Data are located near greatest demand site
- Faster data access and processing
- Growth facilitation
- Improved communications
- Reduced operating costs
- User-friendly interface
- Less danger of a single-point failure
- Processor independence

Disadvantages

- Complexity of management and control
- Technological difficulty
- Security
- Lack of standards
- Increased storage and infrastructure requirements
- Increased training cost
- Costs incurred due to the requirement of duplicated infrastructure

Distributed Processing and Distributed Databases

- **Distributed processing:** Database's logical processing is shared among two or more physically independent sites via network
- **Distributed database:** Stores logically related database over two or more physically independent sites via computer network
 - **Database fragments:** Database composed of many parts in distributed database system

Characteristics of Distributed Management Systems

A DBMS must have at least the following functions to be classified as distributed:

- *Application interface* to interact with the end user, application programs, and other DBMSs within the distributed database
- *Validation to analyse* data requests for syntax correctness
- *Transformation* to decompose complex requests into atomic data request components
- *Query optimization* to find the best access strategy (which database fragments must be accessed by the query, and how must data updates, if any, be synchronized?)
- *Mapping* to determine the data location of local and remote fragments

Characteristics of Distributed Management Systems

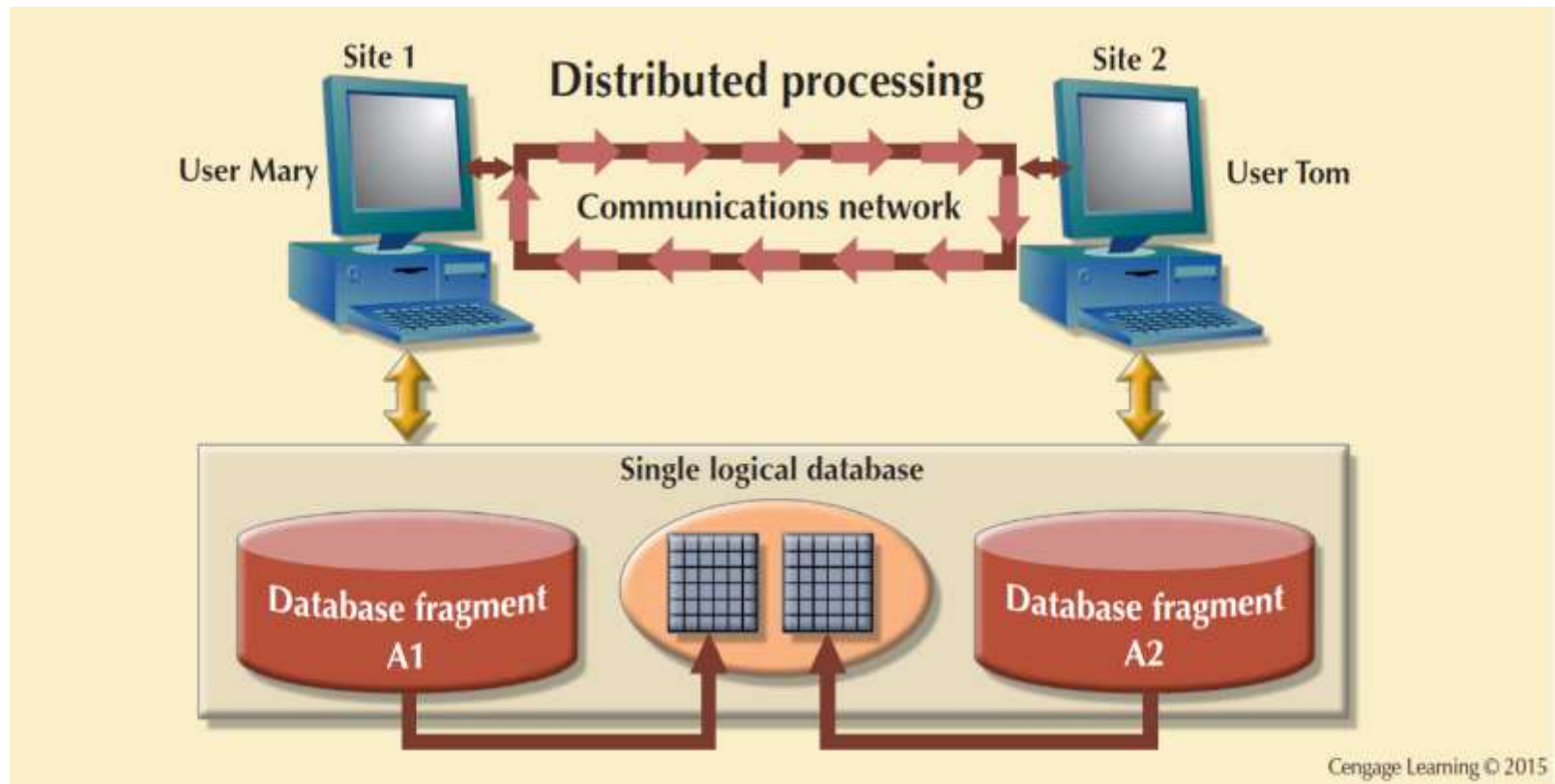
A DBMS must have at least the following functions to be classified as distributed:

- **I/O interface** to read or write data from or to permanent local storage
- **Formatting** to prepare the data for presentation to the end user or to an application program
- **Security** to provide data privacy at both local and remote databases
- **Backup and recovery** to ensure the availability and recoverability of the database in case of a failure
- **DB administration** features for the database administrator
- **Concurrency control** to manage simultaneous data access and to ensure data consistency across database fragments in the DDBMS.
- **Transaction management** to ensure that the data moves from one consistent state to another; this activity includes the synchronization of local and remote transactions as well as transactions across multiple distributed segments

Functions of Distributed DBMS

- Receives the request of an application
- **Validates analyzes**, and decomposes the request
- Maps the request
- Decomposes request into several I/O operations
- Searches and validates data
- Ensures consistency, security, and integrity
- Validates data for specific conditions
- Presents data in required format

Figure 12.4 - A Fully Distributed Database Management System



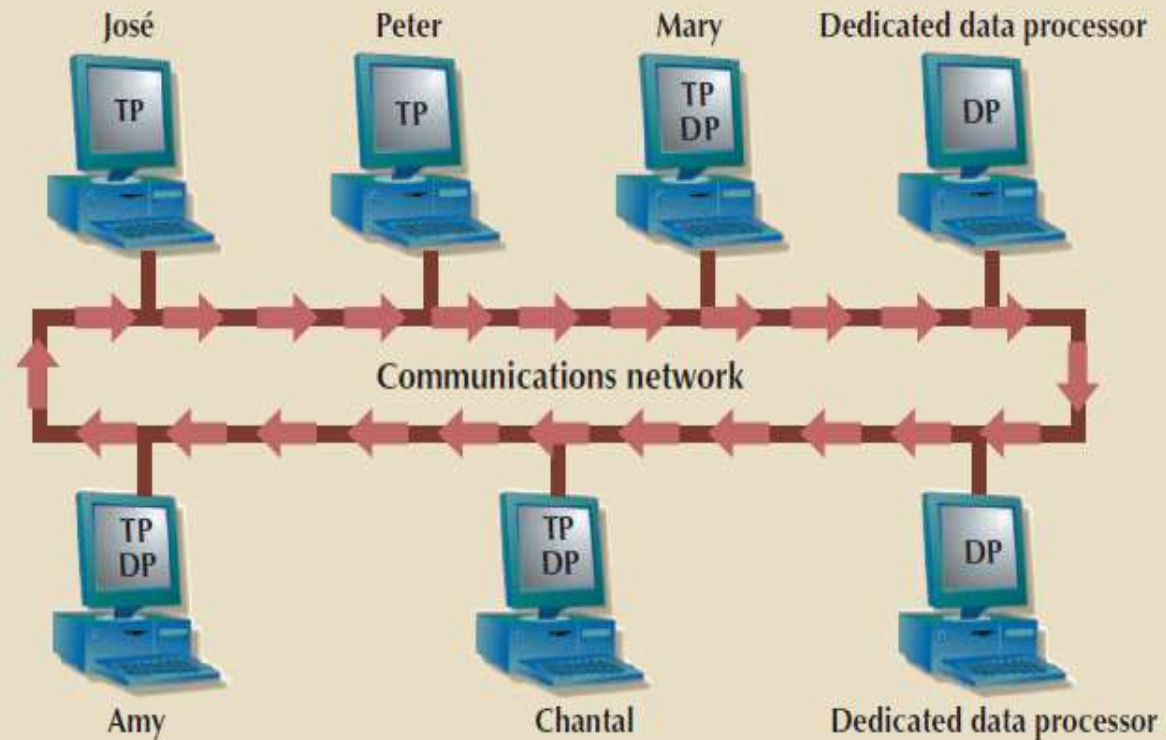
Cengage Learning © 2015

DDBMS Components

- Computer workstations or remote devices
- Network hardware and software components
- Communications media
- **Transaction processor (TP):** Software component of a system that requests data
 - Known as **transaction manager (TM)** or **application processor (AP)**
- **Data processor (DP) or data manager (DM)**
 - **Software** component on a system that stores and retrieves data from its location

DDBMS Components

FIGURE 12.5 DISTRIBUTED DATABASE SYSTEM COMPONENTS



Note: Each TP can access data on any DP, and each DP handles all requests for local data from any TP.

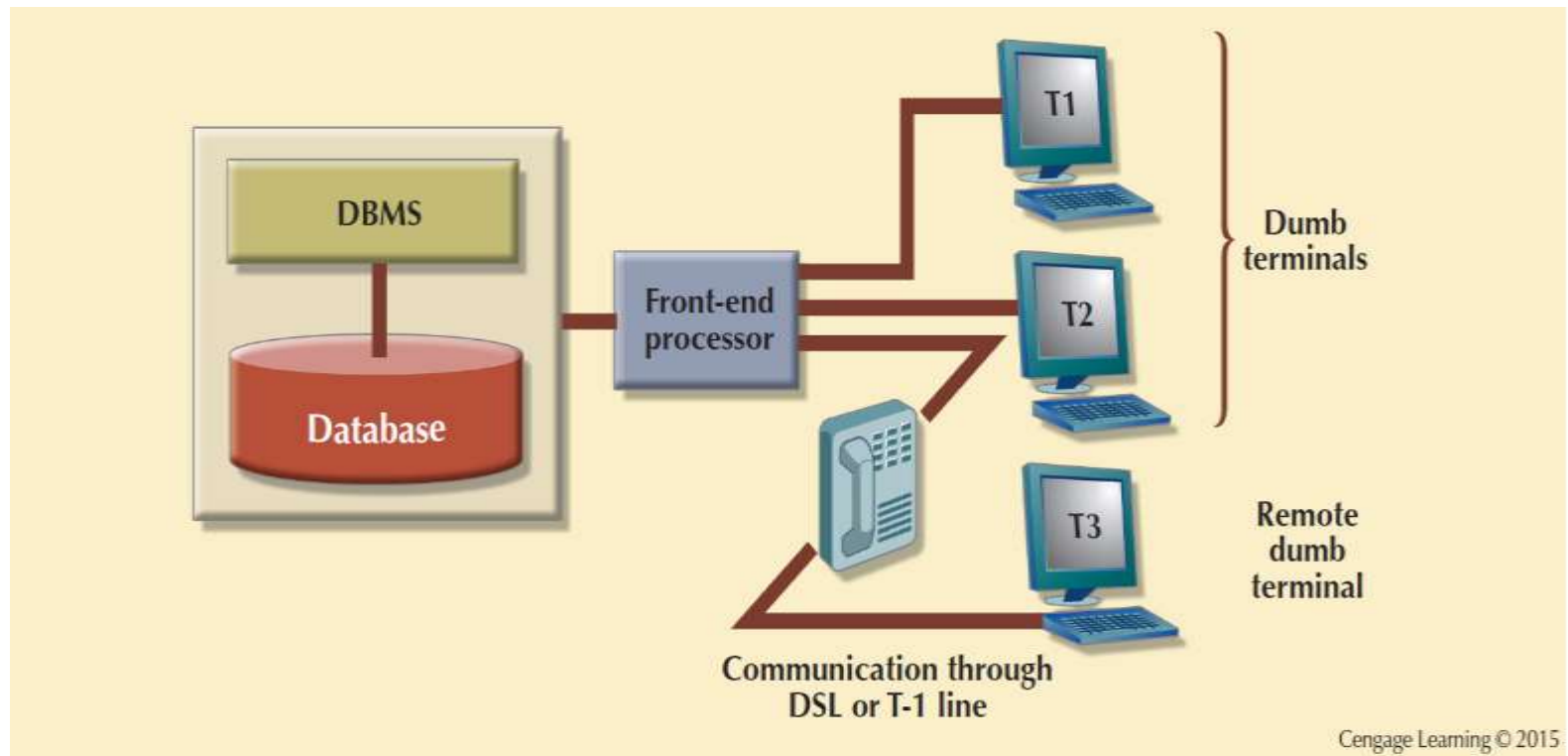
Levels of Data and Process Distribution

- Current database systems can be classified on the basis of how process distribution and data distribution are supported.
 - Single-Site Processing, Single-Site Data
 - Multiple-Site Processing, Single-Site Data
 - Multiple-Site Processing, Multiple-Site Data

Single-Site Processing, Single-Site Data (SPSD)

- Processing is done on a single host computer
- Data stored on host computer's local disk
- Processing restricted on end user's side
- DBMS is accessed by dumb terminals

Figure 12.6 - Single-Site Processing, Single-Site Data (Centralized)

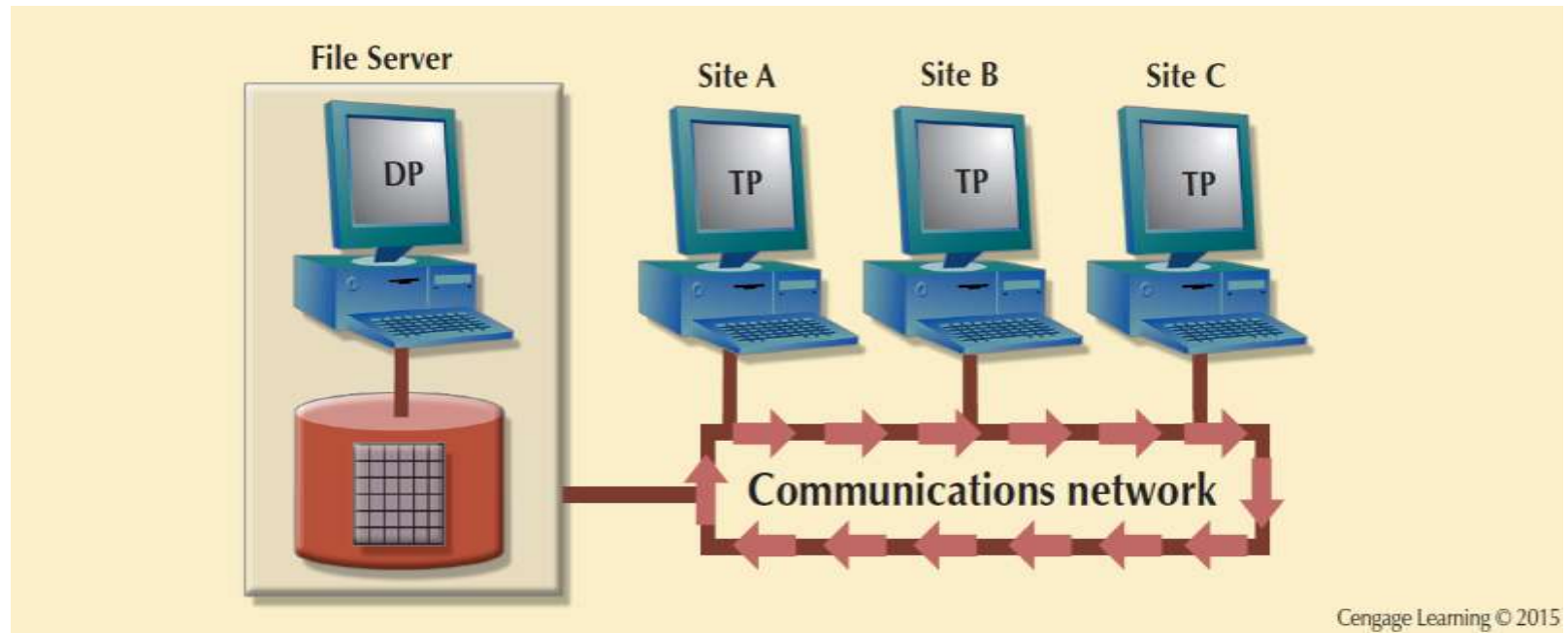


Cengage Learning © 2015

Multiple-Site Processing, Single-Site Data (MPSD)

- Multiple processes run on different computers sharing a single data repository
- Require network file server running conventional applications
 - Accessed through LAN
- **Client/server architecture**
 - Reduces network traffic
 - Processing is distributed
 - Supports data at multiple sites

Figure 12.7 - Multiple-Site Processing, Single-Site Data



Multiple-Site Processing, Multiple-Site Data (MPMD)

- Fully distributed database management system
 - Support multiple data processors and transaction processors at multiple sites
- Classification of DDBMS depending on the level of support for various types of databases
 - **Homogeneous:** Integrate multiple instances of same DBMS over a network
 - **Heterogeneous:** Integrate different types of DBMSs
 - **Fully heterogeneous:** Support different DBMSs, each supporting different data model

Restrictions of DDBMS

- Remote access is provided on a read-only basis
- Restrictions on the number of remote tables that may be accessed in a single transaction
- Restrictions on the number of distinct databases that may be accessed
- Restrictions on the database model that may be accessed

Distributed Database Transparency Features

Distribution
transparency

Transaction
transparency

Failure
transparency

Performance
transparency

Heterogeneity
transparency

Distribution Transparency

- Allows management of physically dispersed database as if centralized
- Levels
 - **Fragmentation transparency**
 - **Location transparency**
 - **Local mapping transparency**

Distribution Transparency

- **Unique fragment:** Each row is unique, regardless of the fragment in which it is located
- Supported by **distributed data dictionary (DDD)** or **distributed data catalog (DDC)**
 - DDC contains the description of the entire database as seen by the database administrator
- **Distributed global schema:** Common database schema to translate user requests into subqueries

Distribution Transparency

FIGURE 12.8 FRAGMENT LOCATIONS



Case 1: The Database Supports Fragmentation Transparency

The query conforms to a nondistributed database query format; that is, it does not specify fragment names or locations. The query reads:

```
SELECT      *
FROM        EMPLOYEE
WHERE       EMP_DOB < '01-JAN-1979';
```

Distribution Transparency

Case 2: The Database Supports Location Transparency

Fragment names must be specified in the query, but the fragment's location is not specified. The query reads:

```
SELECT      *
FROM        E1

WHERE       EMP_DOB < '01-JAN-1979'
UNION
SELECT      *
FROM        E2
WHERE       EMP_DOB < '01-JAN-1979'
UNION
SELECT      *
FROM        E3
WHERE       EMP_DOB < '01-JAN-1979'
```

Distribution Transparency

Case 3: The Database Supports Local Mapping Transparency

Both the fragment name and its location must be specified in the query. Using pseudo-SQL:

```
SELECT      *
FROM        E1 NODE NY
WHERE       EMP_DOB < '01-JAN-1979';
UNION
SELECT      *
FROM        E2 NODE ATL
WHERE       EMP_DOB < '01-JAN-1979';
UNION
SELECT      *
FROM        E3 NODE MIA
WHERE       EMP_DOB < '01-JAN-1979';
```

Distribution Transparency

TABLE 12.4

SUMMARY OF TRANSPARENCY FEATURES

IF THE SQL STATEMENT REQUIRES:

FRAGMENT NAME?	LOCATION NAME?	THEN THE DBMS SUPPORTS	LEVEL OF DISTRIBUTION TRANSPARENCY
Yes	Yes	Local mapping transparency	Low
Yes	No	Location transparency	Medium
No	No	Fragmentation transparency	High

Transaction Transparency

- Ensures database transactions will maintain distributed database's integrity and consistency
- Ensures transaction completed only when all database sites involved complete their part
- Distributed database systems require complex mechanisms to manage transactions

Distributed Requests and Distributed Transactions

Remote request

- Single SQL statement accesses data processed by a single remote database processor

Remote transaction

- Accesses data at single remote site composed of several requests

Distributed transaction

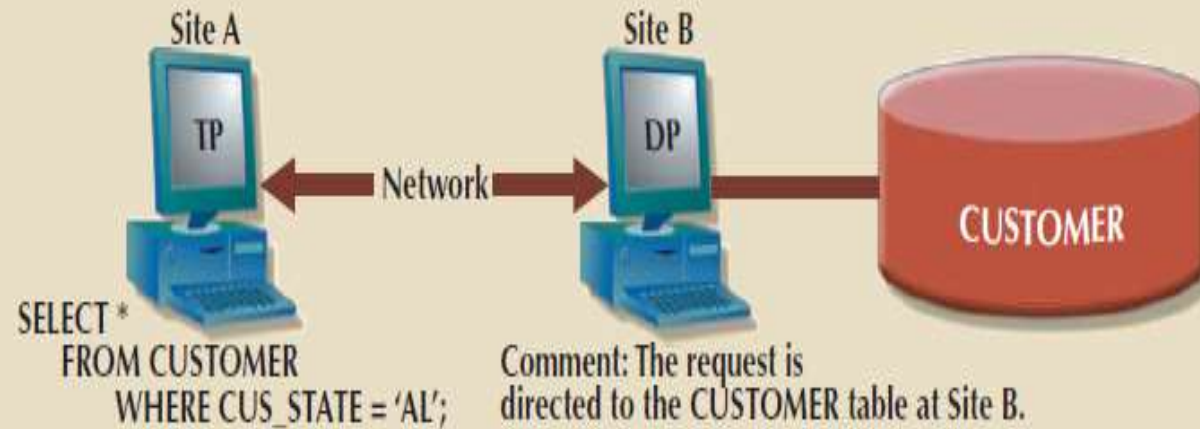
- Requests data from several different remote sites on network

Distributed request

- Single SQL statement references data at several DP sites

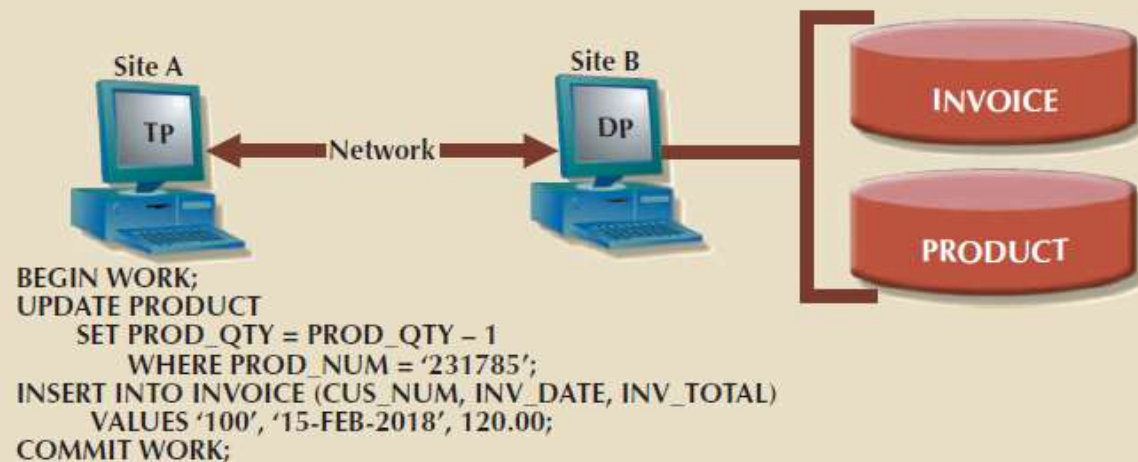
A remote request

FIGURE 12.9 A REMOTE REQUEST



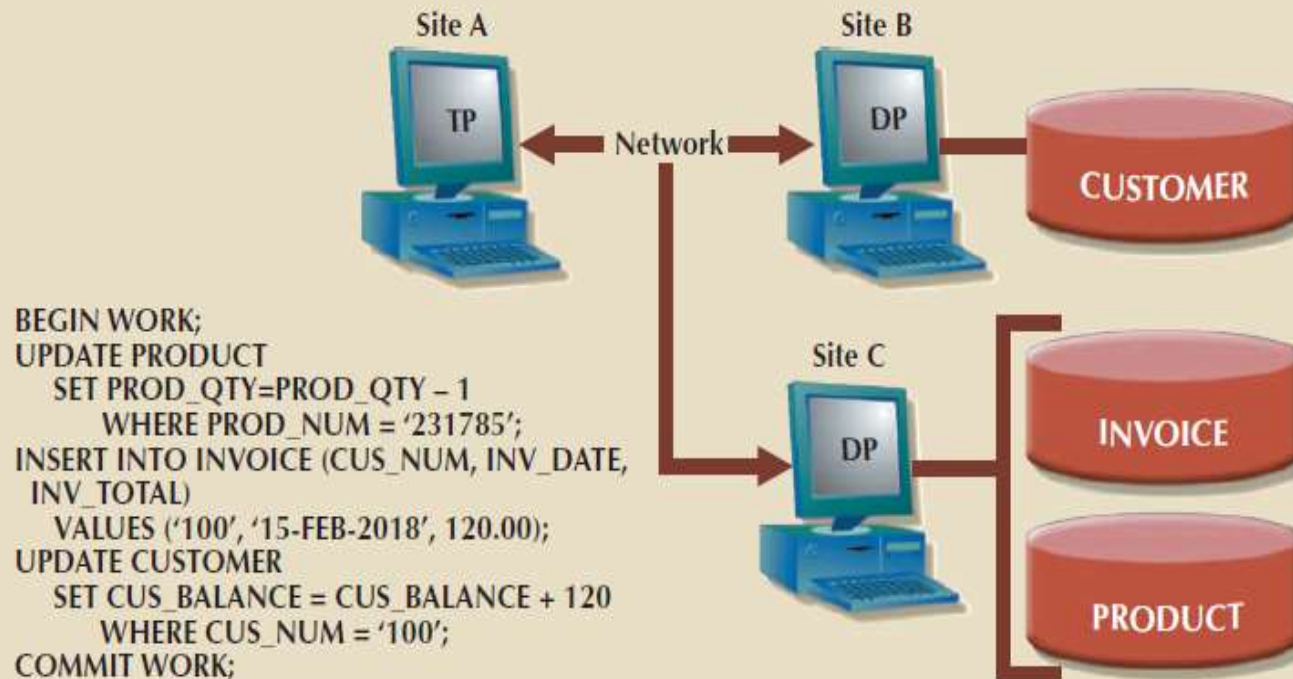
A remote transaction

FIGURE 12.10 A REMOTE TRANSACTION



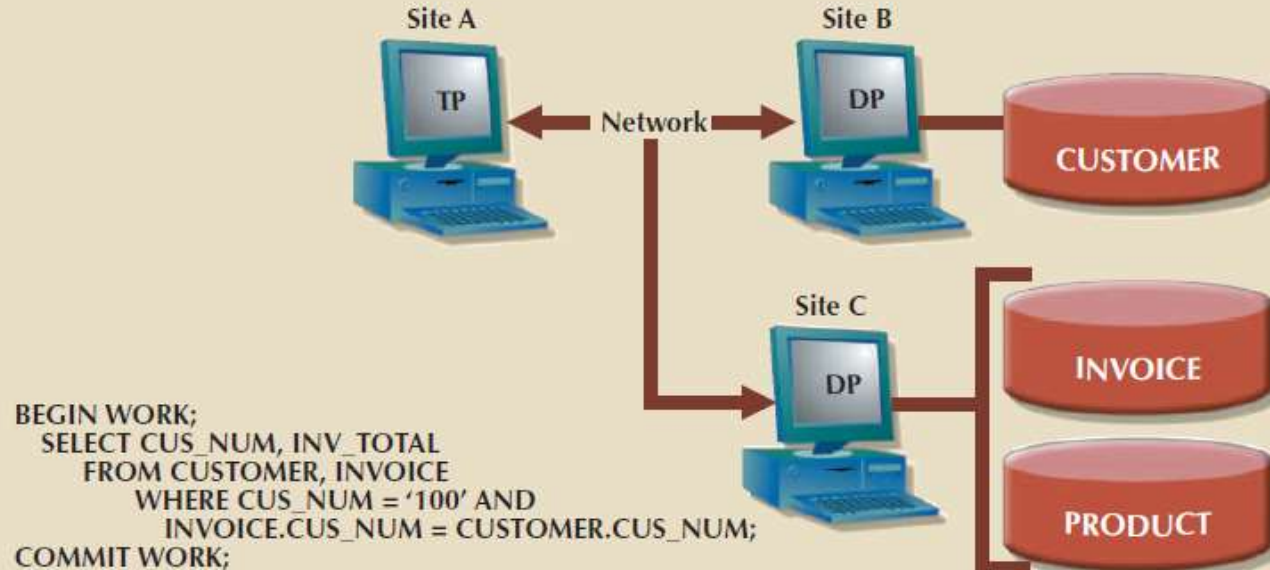
A distributed transaction

FIGURE 12.11 A DISTRIBUTED TRANSACTION



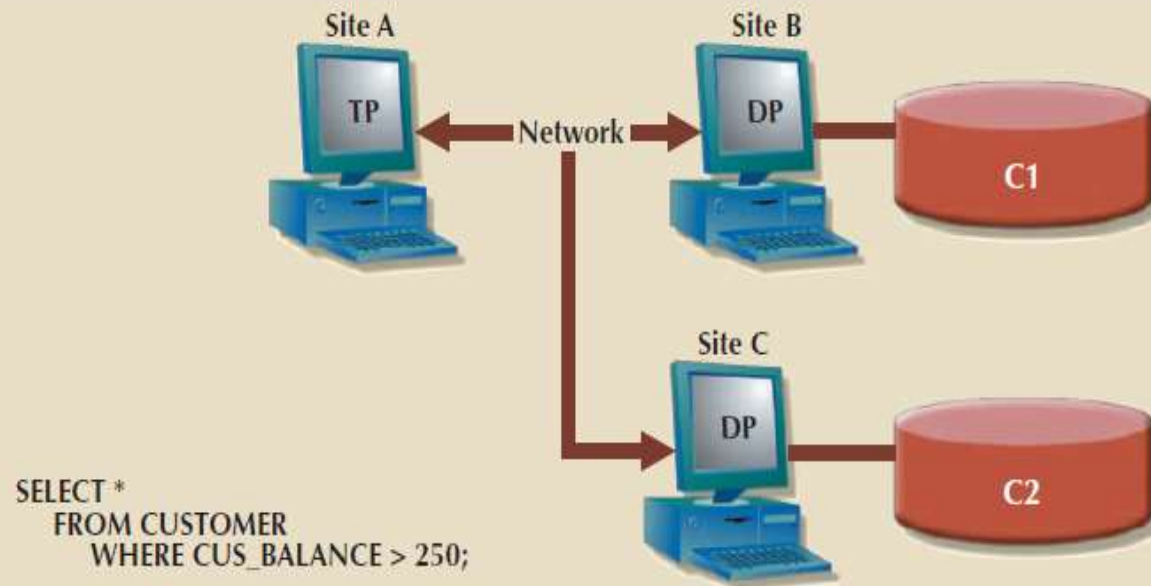
A distributed request

FIGURE 12.12 A DISTRIBUTED REQUEST



A distributed request

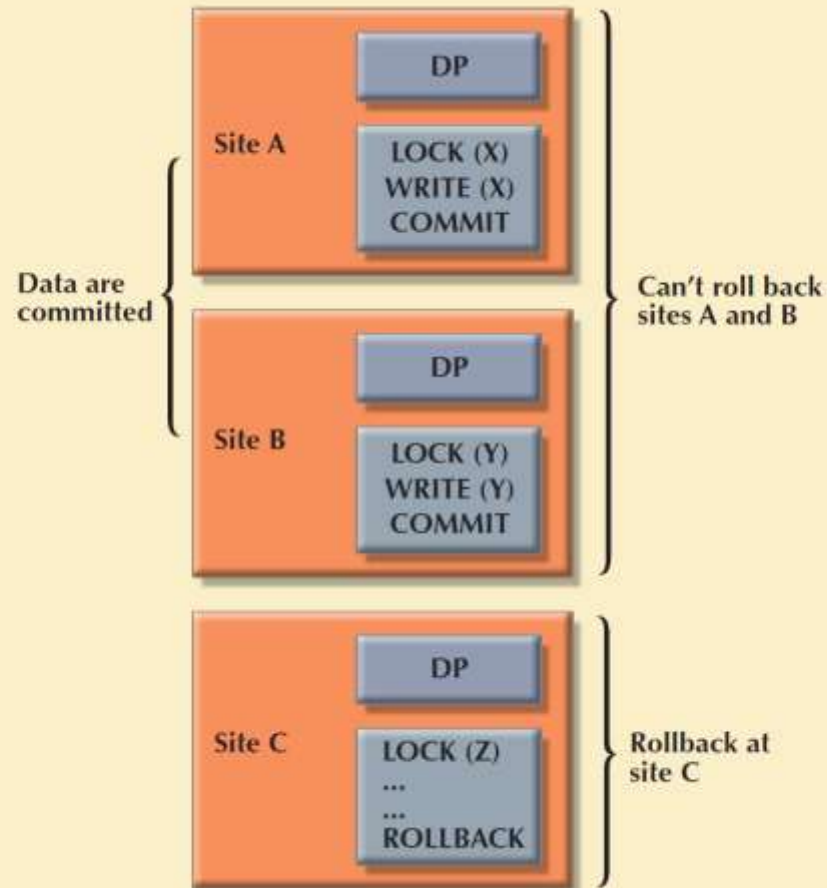
FIGURE 12.13 ANOTHER DISTRIBUTED REQUEST



Distributed Concurrency Control

- Concurrency control is important in distributed databases environment
 - Due to multi-site multiple-process operations that create inconsistencies and deadlocked transactions
- A final COMMIT must not be issued until all sites have committed their parts of the transaction.

Figure 12.14 - The Effect of Premature COMMIT



Cengage Learning © 2015

Two-Phase Commit Protocol (2PC)

- Guarantees if a portion of a transaction operation cannot be committed, all changes made at the other sites will be undone
 - To maintain a consistent database state
- Requires that each DP's transaction log entry be written before database fragment is updated
- **DO-UNDO-REDO protocol:** Roll transactions back and forward with the help of the system's transaction log entries

DO-UNDO-REDO protocol

- The DO-UNDO-REDO protocol defines three types of operations:
- **DO** performs the operation and records the “before” and “after” values in the transaction log.
- **UNDO** reverses an operation, using the log entries written by the DO portion of the sequence.
- **REDO** redoes an operation, using the log entries written by the DO portion of the sequence.

Two-Phase Commit Protocol (2PC)

- **Write-ahead protocol**: Forces the log entry to be written to permanent storage before actual operation takes place
- Defines operations between **coordinator** and **subordinates**
- Phases of implementation
 - Preparation
 - The final COMMIT

Two-Phase Commit Protocol (2PC)

- Phase I: Preparation
- The coordinator sends a PREPARE TO COMMIT message to all subordinates.
- 1. The subordinates receive the message, write the transaction log using the write-ahead protocol, and send an acknowledgment message (YES/PREPARED TO COMMIT or NO/NOT PREPARED) to the coordinator.
- 2. The coordinator makes sure that all nodes are ready to commit, or it aborts the action.
- If all nodes are PREPARED TO COMMIT, the transaction goes to Phase 2.
- If one or more nodes reply NO or NOT PREPARED, the coordinator broadcasts an ABORT message to all subordinates.

Two-Phase Commit Protocol (2PC)

- Phase 2: The Final COMMIT
- 1. The coordinator broadcasts a COMMIT message to all subordinates and waits for the replies.
- 2. Each subordinate receives the COMMIT message and then updates the database using the DO protocol.
- 3. The subordinates reply with a COMMITTED or NOT COMMITTED message to the coordinator.
- If one or more subordinates do not commit, the coordinator sends an ABORT message, thereby forcing them to UNDO all changes.

Performance and Failure Transparency

- **Performance transparency:** Allows a DDBMS to perform as if it were a centralized database
- **Failure transparency:** Ensures the system will operate in case of network failure
- Considerations for resolving requests in a distributed data environment
 - Data distribution
 - Data replication
 - **Replica transparency:** DDBMS's ability to hide multiple copies of data from the user

Performance and Failure Transparency

- Network and node availability
 - **Network latency:** delay imposed by the amount of time required for a data packet to make a round trip
 - **Network partitioning:** delay imposed when nodes become suddenly unavailable due to a network failure

Distributed Database Design

Data fragmentation

- How to partition database into fragments

Data replication

- Which fragments to replicate

Data allocation

- Where to locate those fragments and replicas

Data Fragmentation

- Breaks single object into many segments
 - Information is stored in distributed data catalog (DDC)
- Strategies
 - **Horizontal fragmentation:** Division of a relation into subsets (fragments) of tuples (rows)
 - **Vertical fragmentation:** Division of a relation into attribute (column) subsets
 - **Mixed fragmentation:** Combination of horizontal and vertical strategies

Data Replication

- Data copies stored at multiple sites served by a computer network
- **Mutual consistency rule:** Replicated data fragments should be identical
- Styles of replication
 - Push replication
 - Pull replication
- Helps restore lost data

Data Replication Scenarios

Fully replicated database

- Stores multiple copies of each database fragment at multiple sites

Partially replicated database

- Stores multiple copies of some database fragments at multiple sites

Unreplicated database

- Stores each database fragment at a single site

Data Allocation Strategies

Centralized data allocation

- Entire database stored at one site

Partitioned data allocation

- Database is divided into two or more disjointed fragments and stored at two or more sites

Replicated data allocation

- Copies of one or more database fragments are stored at several sites

The CAP Theorem

- CAP stands for:
 - Consistency
 - Availability
 - Partition tolerance
- **Basically available, soft state, eventually consistent (BASE)**
 - Data changes are not immediate but propagate slowly through the system until all replicas are consistent

Table 12.8 - Distributed Database Spectrum

TABLE 12.8

DISTRIBUTED DATABASE SPECTRUM

DBMS TYPE	CONSISTENCY	AVAILABILITY	PARTITION TOLERANCE	TRANSACTION MODEL	TRADE-OFF
Centralized DBMS	High	High	N/A	ACID	No distributed data processing
Relational DBMS	High	Relaxed	High	ACID (2PC)	Sacrifices availability to ensure consistency and isolation
NoSQL DDBMS	Relaxed	High	High	BASE	Sacrifices consistency to ensure availability
NewSQL DDBMS	High	High	Relaxed	ACID	Sacrifices partition tolerance to ensure transaction consistency and availability

C. J. Date's Twelve Commandments for Distributed Databases

- Local site independence
- Central site independence
- Failure independence
- Location transparency
- Fragmentation transparency
- Replication transparency

C. J. Date's Twelve Commandments for Distributed Databases

- Distributed query processing
- Distributed transaction processing
- Hardware independence
- Operating system independence
- Network independence
- Database independence

Database systems ITCS2322

Chapter 13 : Business Intelligence and Data Warehouses

اعداد : دكتور محمد عبد اللطيف راضي

Learning Objectives

- In this chapter, students will learn:
 - How business intelligence provides a comprehensive business decision support framework
 - About business intelligence architecture, its evolution, and reporting styles
 - About the relationship and differences between operational data and decision support data
 - What a data warehouse is and how to prepare data for one

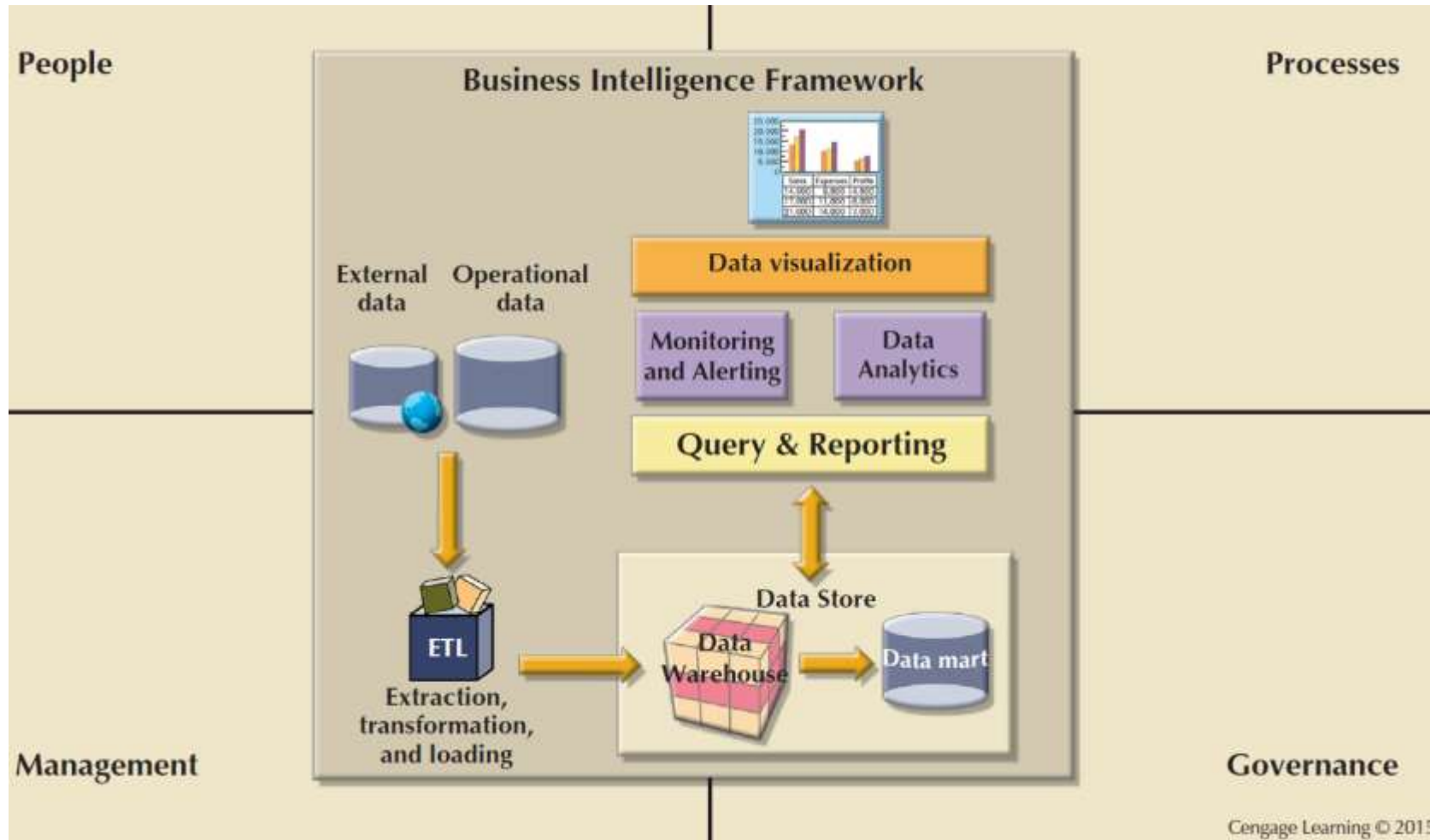
Learning Objectives

- In this chapter, students will learn:
 - What star schemas are and how they are constructed
 - About data analytics, data mining, and predictive analytics
 - About online analytical processing (OLAP)
 - How SQL extensions are used to support OLAP-type data manipulations

Business Intelligence (BI)

- Comprehensive, cohesive, integrated set of tools and processes
 - Captures, collects, integrates, stores, and analyzes data
- Purpose - Generate and present information to support business decision making
- Allows a business to transform:
 - Data into information
 - Information into knowledge
 - Knowledge into wisdom

Figure 13.1 - Business Intelligence Framework



Cengage Learning © 2015

Basic BI Architectural Components

TABLE 13.2

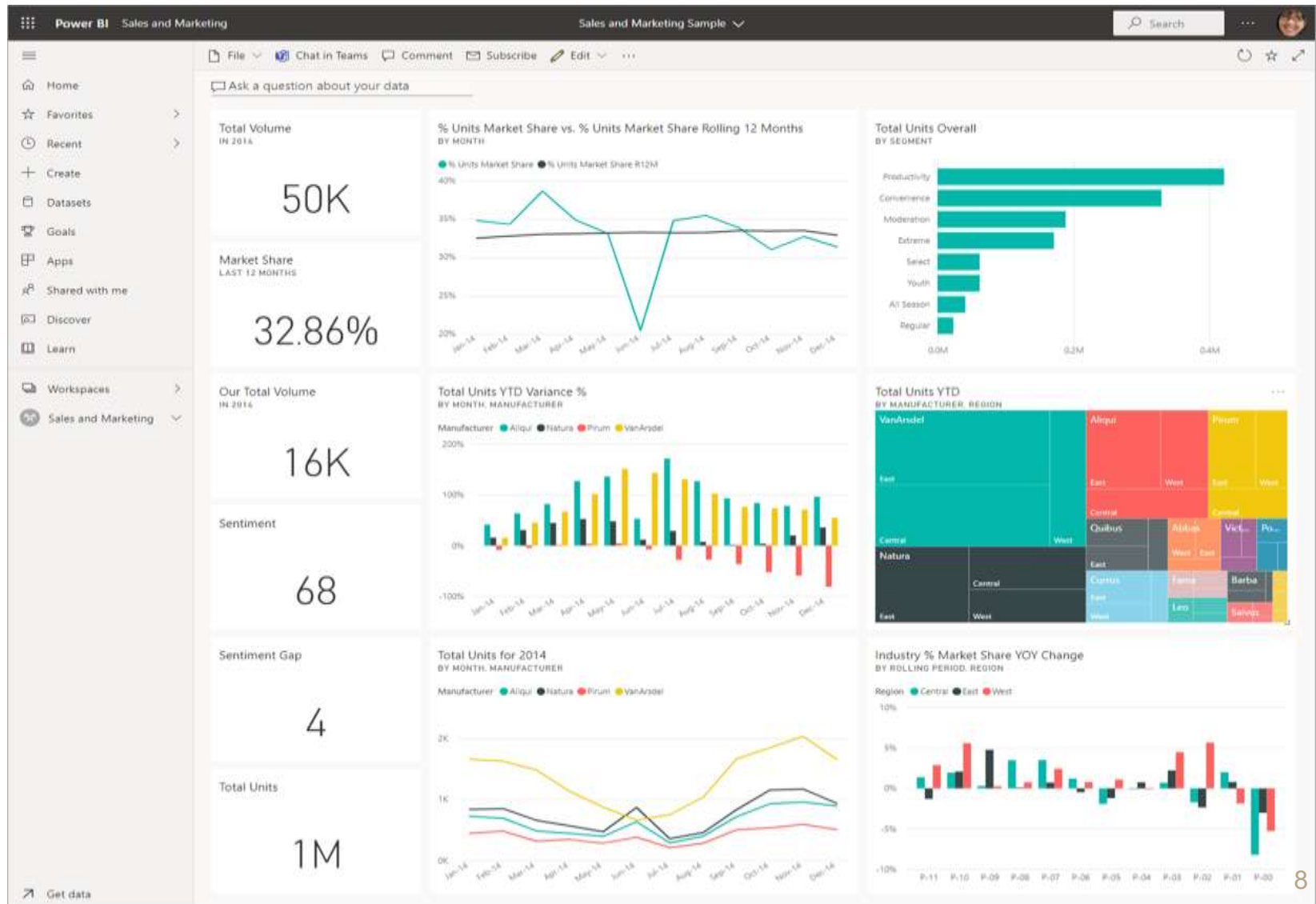
BASIC BI ARCHITECTURAL COMPONENTS

COMPONENT	DESCRIPTION
ETL tools	Data extraction, transformation, and loading (ETL) tools collect, filter, integrate, and aggregate internal and external data to be saved into a data store optimized for decision support.
Data store	The data store is optimized for decision support and is generally represented by a <i>data warehouse</i> or a <i>data mart</i> . The data is stored in structures that are optimized for data analysis and query speed.
Query and reporting	This component performs data selection and retrieval, and it is used by the data analyst to create queries that access the database and create the required reports.
Data visualization	This component presents data to the end user in a variety of meaningful and innovative ways. This tool helps the end user select the most appropriate presentation format, such as summary reports, maps, pie or bar graphs, mixed graphs, and static or interactive dashboards.
Data monitoring and alerting	This component allows real-time monitoring of business activities. The BI system will present concise information in a single integrated view. This integrated view could include specific metrics about the system performance or activities, such as number of orders placed in the last four hours, number of customer complaints by product by month, and total revenue by region. Alerts can be placed on a given metric; once the value of a metric goes below or above a certain baseline, the system will perform a given action, such as emailing shop floor managers, presenting visual alerts, or starting an application.
Data analytics	This component performs data analysis and data-mining tasks using the data in the data store. This tool advises the user as to which data analysis tool to select and how to build a reliable business data model. Business models are generated by special algorithms that identify and enhance the understanding of business situations and problems.

Business Intelligence Tools

- Dashboards and business activity monitoring
 - **Dashboards:** Shows key business performance indicators in a single integrated view
- **Portals:** Integrate data using web browser from multiple sources into a single webpage
- Data analysis and reporting tools
- Data-mining tools
- Data warehouses (DW)
- OLAP(**Online analytical processing**) tools and data visualization

Dashboards



Practices to Manage Data

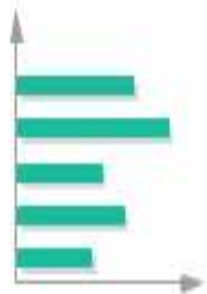
- **Master data management (MDM):**
Collection of concepts, techniques, and processes for identification, definition, and management of data elements
- **Governance:** Method of government for controlling business health and for consistent decision making
- **Key performance indicators (KPI):**
Numeric or scale-based measurements that assess company's effectiveness in reaching its goals

Practices to Manage Data

- **Data visualization:** Abstracting data to provide information in a visual format
 - Enhances the user's ability to efficiently comprehend the meaning of the data
 - Techniques
 - Pie charts and bar charts
 - Line graphs
 - Scatter plots
 - Gantt charts
 - Heat maps



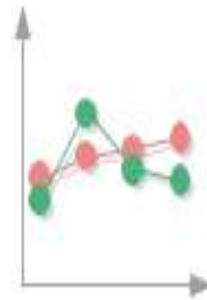
Pie



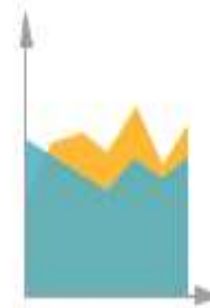
Bar



Column



Line



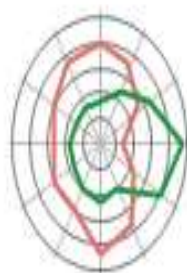
Area



Doughnut



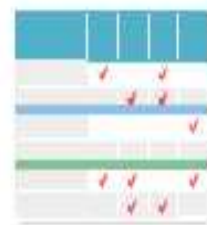
Bubble Chart



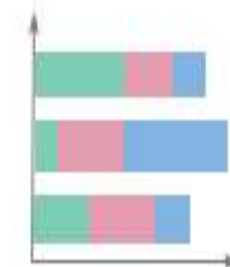
Spider and Radar



Scatter



Comparison Chart



Stacked bar chart



Gauges

Reporting Styles of a Modern BI System

Advanced reporting

Monitoring and
alerting

Advanced data
analytics

Business Intelligence Benefits

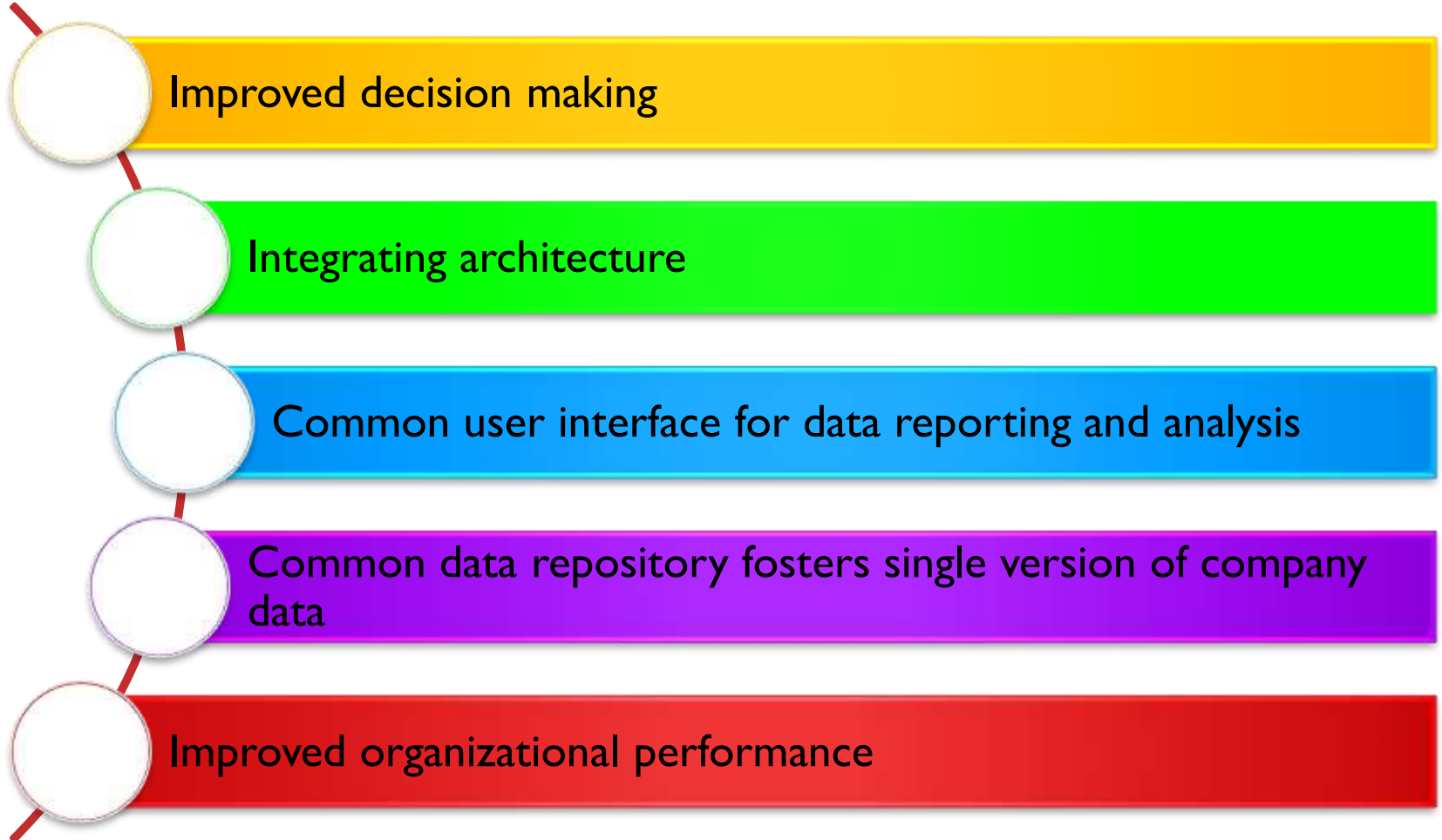


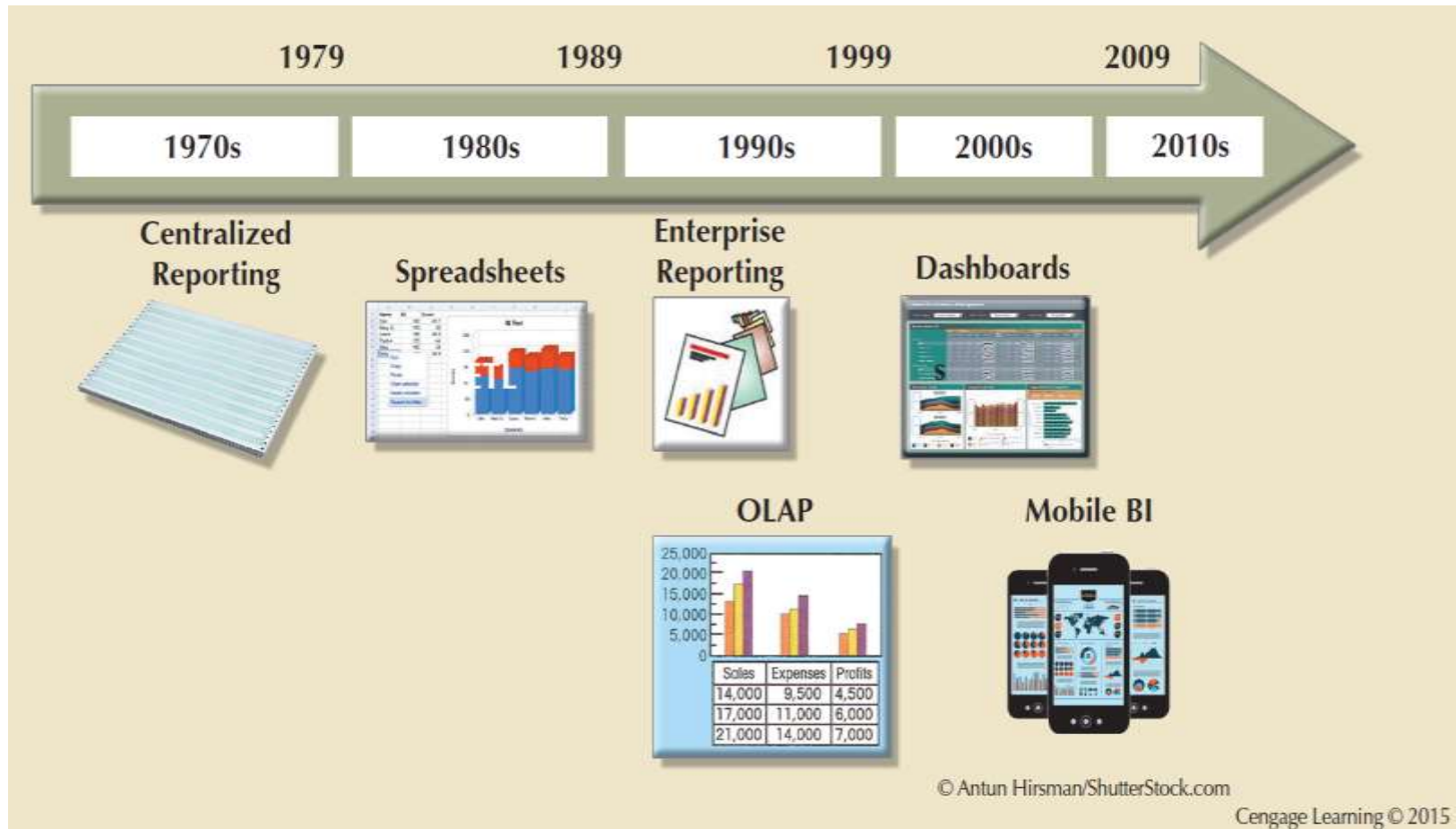
Table 13.4 - Business Intelligence Evolution

BUSINESS INTELLIGENCE EVOLUTION					
SYSTEM TYPE	DATA SOURCE	DATA EXTRACTION/ INTEGRATION PROCESS	DATA STORE	END-USER QUERY TOOL	END USER PRESENTATION TOOL
Traditional mainframe-based online transaction processing (OLTP)	Operational data	None Reports read and summarized data directly from operational data	None Temporary files used for reporting purposes	Very basic Predefined reporting formats Basic sorting, totaling, and averaging	Very basic Menu-driven, predefined reports, text and numbers only
Managerial information system (MIS)	Operational data	Basic extraction and aggregation Read, filter, and summarize operational data into intermediate data store	Lightly aggregated data in RDBMS	Same as above, in addition to some ad hoc reporting using SQL	Same as above, in addition to some ad hoc columnar report definitions
First-generation departmental decision support system (DSS)	Operational data External data	Data extraction and integration process populates DSS data store Run periodically	First DSS database generation Usually RDBMS	Query tool with some analytical capabilities and predefined reports	Spreadsheet style Advanced presentation tools with plotting and graphics capabilities

Table 13.4 - Business Intelligence Evolution

SYSTEM TYPE	DATA SOURCE	DATA EXTRACTION/ INTEGRATION PROCESS	DATA STORE	END-USER QUERY TOOL	END USER PRESENTATION TOOL
First-generation BI	Operational data External data	Advanced data extraction and integration Access diverse data sources, filters, aggregations, classifications, scheduling, and conflict resolution	Data warehouse RDBMS technology Optimized for query purposes Star schema model	Same as above	Same as above, in addition to multidimensional presentation tools with drill-down capabilities
Second-generation BI Online analytical processing (OLAP)	Same as above	Same as above	Data warehouse stores data in MDBMS Cubes with multiple dimensions	Adds support for end-user-based data analytics	Same as above, but uses cubes and multidimensional matrixes; limited by cube size Dashboards Scorecards Portals
Third-generation Mobile, cloud-based, and Big Data	Same as above Includes social media and machine-generated data	Same as above Cloud-based	Same as above Cloud-based Hadoop and NoSQL databases	Advanced analytics Flexible ad hoc interactions via data visualization	Mobile devices: smartphones and tablets

Figure 13.3 - Evolution of BI Information Dissemination Formats



Business Intelligence Technology Trends

Data storage improvements

Business intelligence appliances

Business intelligence as a service

Big Data analytics

Personal analytics

Decision Support Data

- Effectiveness of BI depends on quality of data gathered at operational level
- Operational data
 - Seldom well-suited for decision support tasks
 - Stored in relational database with highly normalized structures
 - Optimized to support transactions representing daily operations

Decision Support Data

- Differ from operational data in:
 - **Time span:** decision support data tends to cover a longer time frame
- **Granularity:** Decision support data must be presented at different levels of aggregation, from highly summarized to nearly atomic.
- **Dimensionality:** Operational data focuses on representing individual transactions rather than the effects of the transactions over time

Table 13.5 - Contrasting Operational and Decision Support Data Characteristics

CHARACTERISTIC	OPERATIONAL DATA	DECISION SUPPORT DATA
Data currency	Current operations Real-time data	Historic data Snapshot of company data Time component (week/month/year)
Granularity	Atomic-detailed data	Summarized data
Summarization level	Low; some aggregate yields	High; many aggregation levels
Data model	Highly normalized Mostly relational DBMSs	Non-normalized Complex structures Some relational, but mostly multidimensional DBMSs
Transaction type	Mostly updates	Mostly query
Transaction volumes	High-update volumes	Periodic loads and summary calculations
Transaction speed	Updates are critical	Retrievals are critical
Query activity	Low to medium	High
Query scope	Narrow range	Broad range
Query complexity	Simple to medium	Very complex
Data volumes	Hundreds of gigabytes	Terabytes to petabytes

Decision Support Database Requirements

- Database schema
 - Must support complex, non-normalized data representations
 - Data must be aggregated and summarized
 - Queries must be able to extract multidimensional time slices

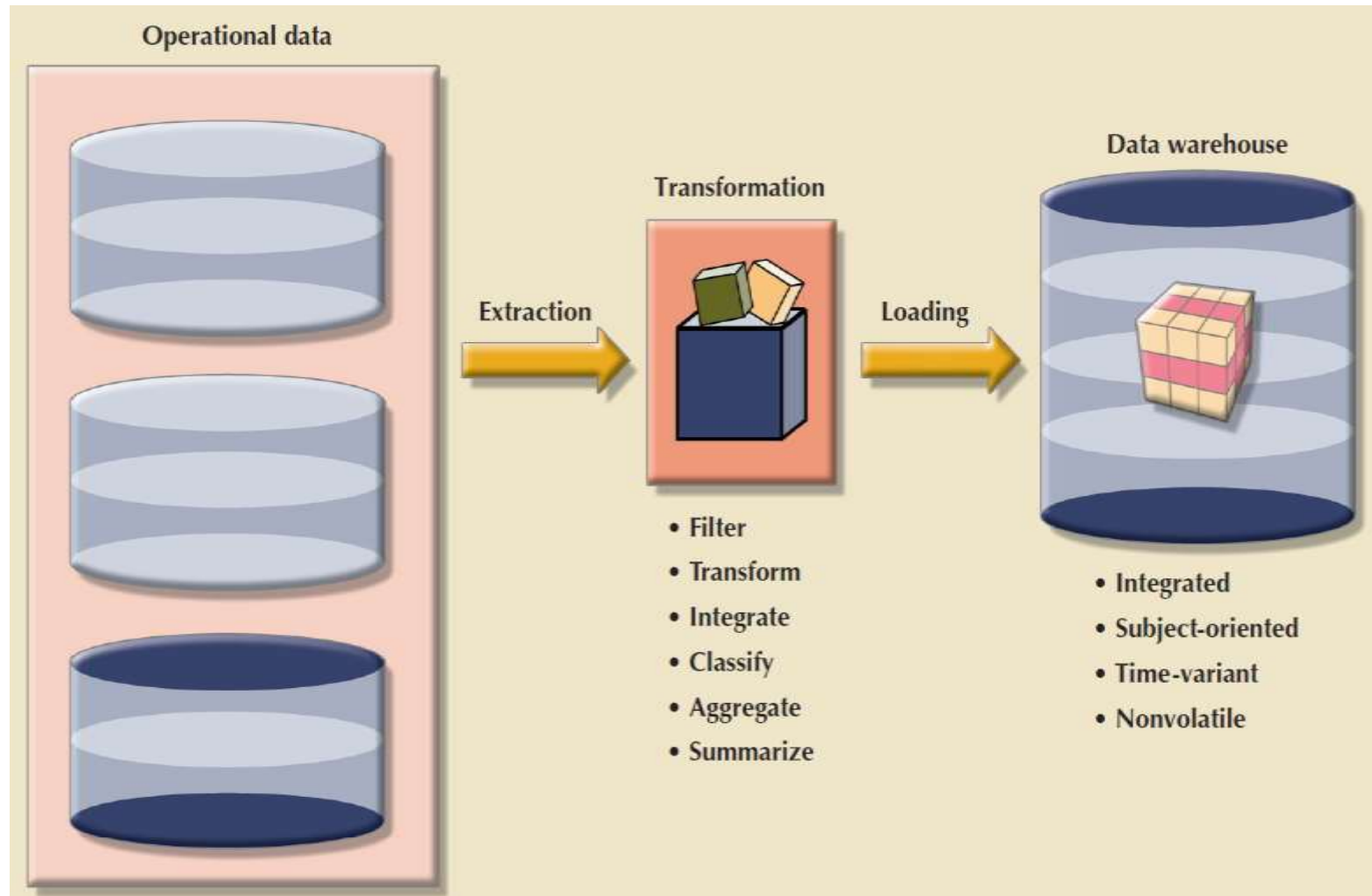
Decision Support Database Requirements

- Data extraction and loading
 - Allow batch and scheduled data extraction
 - Support different data sources and check for inconsistent data or data validation rules
 - Support advanced integration, aggregation, and classification
- Database size should support:
 - **Very large databases (VLDBs)**
 - Advanced storage technologies
 - Multiple-processor technologies

Table 13.8 - Characteristics of Data Warehouse Data and Operational Database Data

CHARACTERISTIC	OPERATIONAL DATABASE DATA	DATA WAREHOUSE DATA
Integrated	Similar data can have different representations or meanings. For example, Social Security numbers may be stored as ###-##-#### or as #####, and a given condition may be labeled as T/F or 0/1 or Y/N. A sales value may be shown in thousands or in millions.	Provide a unified view of all data elements with a common definition and representation for all business units.
Subject-oriented	Data are stored with a functional, or process, orientation. For example, data may be stored for invoices, payments, and credit amounts.	Data are stored with a subject orientation that facilitates multiple views of the data and decision making. For example, sales may be recorded by product, division, manager, or region.
Time-variant	Data are recorded as current transactions. For example, the sales data may be the sale of a product on a given date, such as \$342.78 on 12-MAY-2014.	Data are recorded with a historical perspective in mind. Therefore, a time dimension is added to facilitate data analysis and various time comparisons.
Nonvolatile	Data updates are frequent and common. For example, an inventory amount changes with each sale. Therefore, the data environment is fluid.	Data cannot be changed. Data are added only periodically from historical systems. Once the data are properly stored, no changes are allowed. Therefore, the data environment is relatively static.

Figure 13.5 - The ETL Process



Data Marts

- Small, single-subject data warehouse subset
- Provide decision support to a small group of people
- Benefits over data warehouses
 - Lower cost and shorter implementation time
 - Technologically advanced
 - Inevitable people issues

Table 13.9 - Twelve Rules for a Data Warehouse

RULE NO.	DESCRIPTION
1	The data warehouse and operational environments are separated.
2	The data warehouse data are integrated.
3	The data warehouse contains historical data over a long time.
4	The data warehouse data are snapshot data captured at a given point in time.
5	The data warehouse data are subject oriented.
6	The data warehouse data are mainly read-only with periodic batch updates from operational data. No online updates are allowed.

Table 13.9 - Twelve Rules for a Data Warehouse

RULE NO.	DESCRIPTION
7	The data warehouse development life cycle differs from classical systems development. Data warehouse development is data-driven; the classical approach is process-driven.
8	The data warehouse contains data with several levels of detail: current detail data, old detail data, lightly summarized data, and highly summarized data.
9	The data warehouse environment is characterized by read-only transactions to very large data sets. The operational environment is characterized by numerous update transactions to a few data entities at a time.
10	The data warehouse environment has a system that traces data sources, transformations, and storage.
11	The data warehouse's metadata are a critical component of this environment. The metadata identify and define all data elements. The metadata provide the source, transformation, integration, storage, usage, relationships, and history of each data element.
12	The data warehouse contains a chargeback mechanism for resource usage that enforces optimal use of the data by end users.

Star Schema

- Data-modeling technique
- Maps multidimensional decision support data into a relational database
- Creates the near equivalent of multidimensional database schema from existing relational database
- Yields an easily implemented model for multidimensional data analysis

Components of Star Schemas

Facts

- Numeric values that represent a specific business aspect

Dimensions

- Qualifying characteristics that provide additional perspectives to a given fact

Attributes

- Used to search, filter, and classify facts
- **Slice and dice:** Ability to focus on slices of the data cube for more detailed analysis

Attribute hierarchy

- Provides a top-down data organization

Facts

- Are numeric measurements (values) that represent a specific business aspect or activity.
- For example, sales figures are numeric measurements that represent product and service sales.
- Facts commonly used in business data analysis are units, costs, prices, and revenues.
- The fact table contains facts that are linked through their dimensions.
- Facts are normally stored in a fact table that is the centre of the star schema.
- The fact table is updated periodically with data from operational databases.

Dimensions

- Dimensions are qualifying characteristics that provide additional perspectives to a given fact.
- For instance, sales might be compared by product from region to region and from one time period to the next.
- The kind of problem typically addressed by a BI system might be to compare the sales of unit X by region for the first quarters of 2006 through 2016.
- In that example, sales have product, location, and time dimensions.
- Such dimensions are normally stored in dimension tables.

Attributes

- Each dimension table contains attributes.
- Attributes are often used to search, filter, or classify facts.
- *Dimensions provide descriptive characteristics about the facts through their attributes.*
- Therefore, the data warehouse designer must define common business attributes that will be used by the data analyst to narrow a search, group information, or
- describe dimensions.
- Using a sales example, some possible attributes for each dimension are illustrated in Table 13.10

Attributes

TABLE 13.10

POSSIBLE ATTRIBUTES FOR SALES DIMENSIONS

DIMENSION NAME	DESCRIPTION	POSSIBLE ATTRIBUTES
Location	Anything that provides a description of the location—for example, Nashville, Store 101, South Region, and TN	Region, state, city, store, and so on
Product	Anything that provides a description of the product sold—for example, hair care product, shampoo, Natural Essence brand, 5.5-oz. bottle, and blue liquid	Product type, product ID, brand, package, presentation, color, size, and so on
Time	Anything that provides a time frame for the sales fact—for example, the year 2018, the month of July, the date 07/29/2018, and the time 4:46 p.m.	Year, quarter, month, week, day, time of day, and so on

Attribute Hierarchies

- Attributes within dimensions can be ordered in a well-defined attribute hierarchy.
- The attribute hierarchy provides a top-down data organization that is used for two main purposes:
- Aggregation and drill-down/roll-up data analysis.
- For example, Figure 13.8 shows how the location dimension attributes can be organized in a hierarchy by region, state, city, and store.

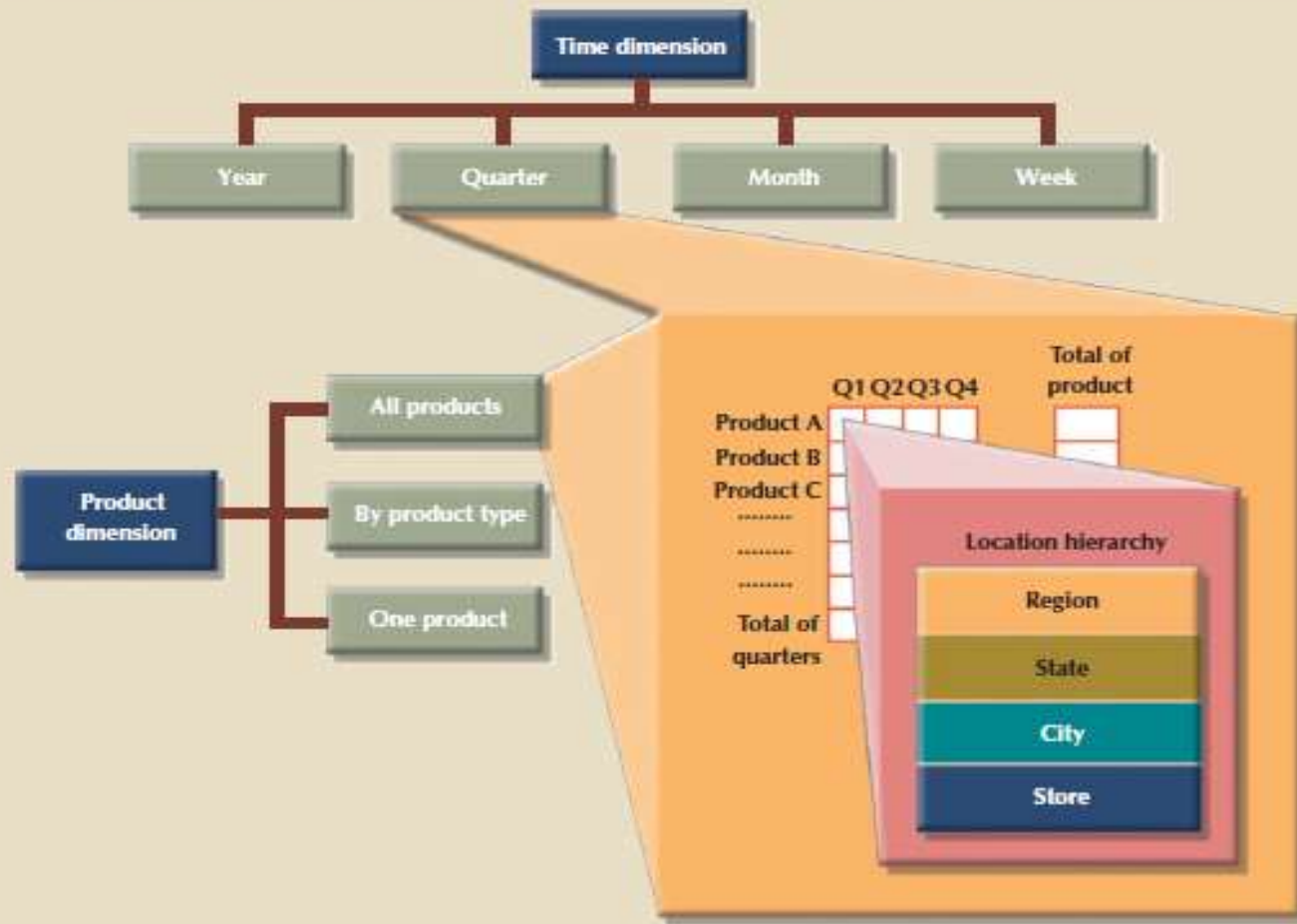
Attribute Hierarchies

FIGURE 13.8 LOCATION ATTRIBUTE HIERARCHY



Attribute Hierarchies

FIGURE 13.9 ATTRIBUTE HIERARCHIES IN MULTIDIMENSIONAL ANALYSIS



Star Schema Representation

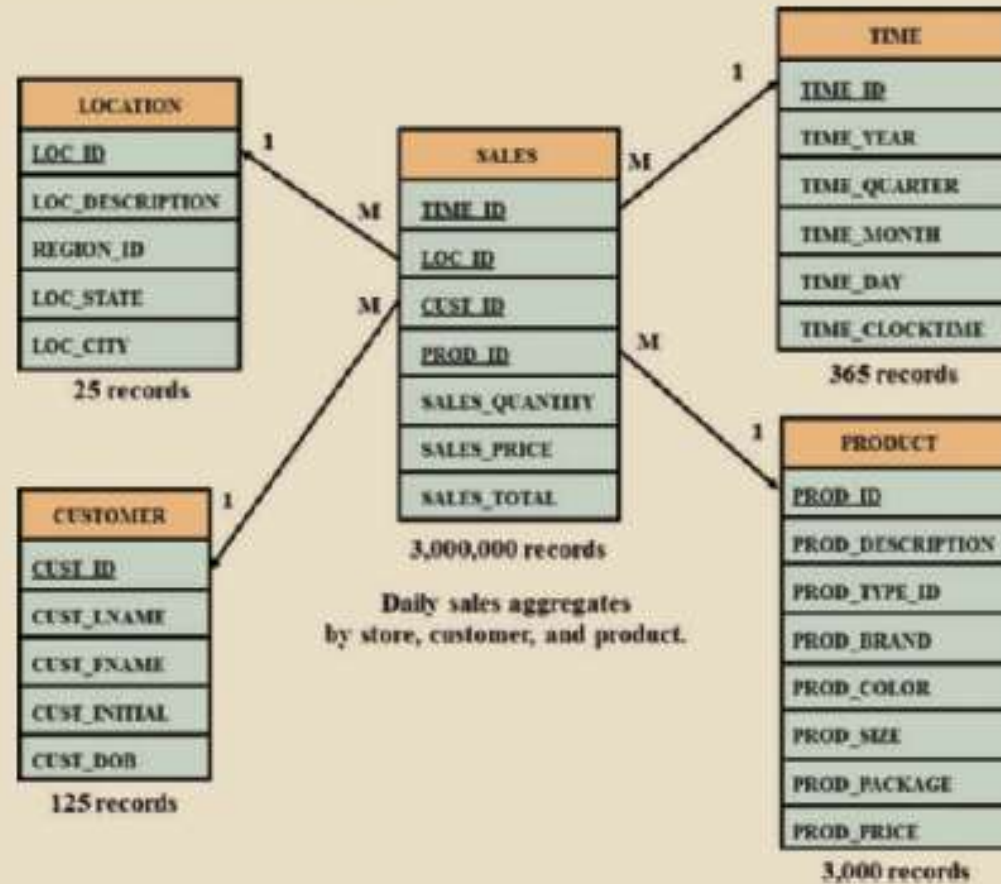
- Facts and dimensions represented by physical tables in data warehouse database
- Many-to-one (M:1) relationship between fact table and each dimension table
- In other words, many fact rows are related to each dimension row
- Fact and dimension tables
 - Related by foreign keys
 - Subject to primary and foreign key constraints

Star Schema Representation

- Primary key of a fact table
 - Is a composite primary key because the fact table is related to many dimension tables
 - Always formed by combining the foreign keys pointing to the related dimension tables

Star Schema Representation

FIGURE 13.10 STAR SCHEMA FOR SALES



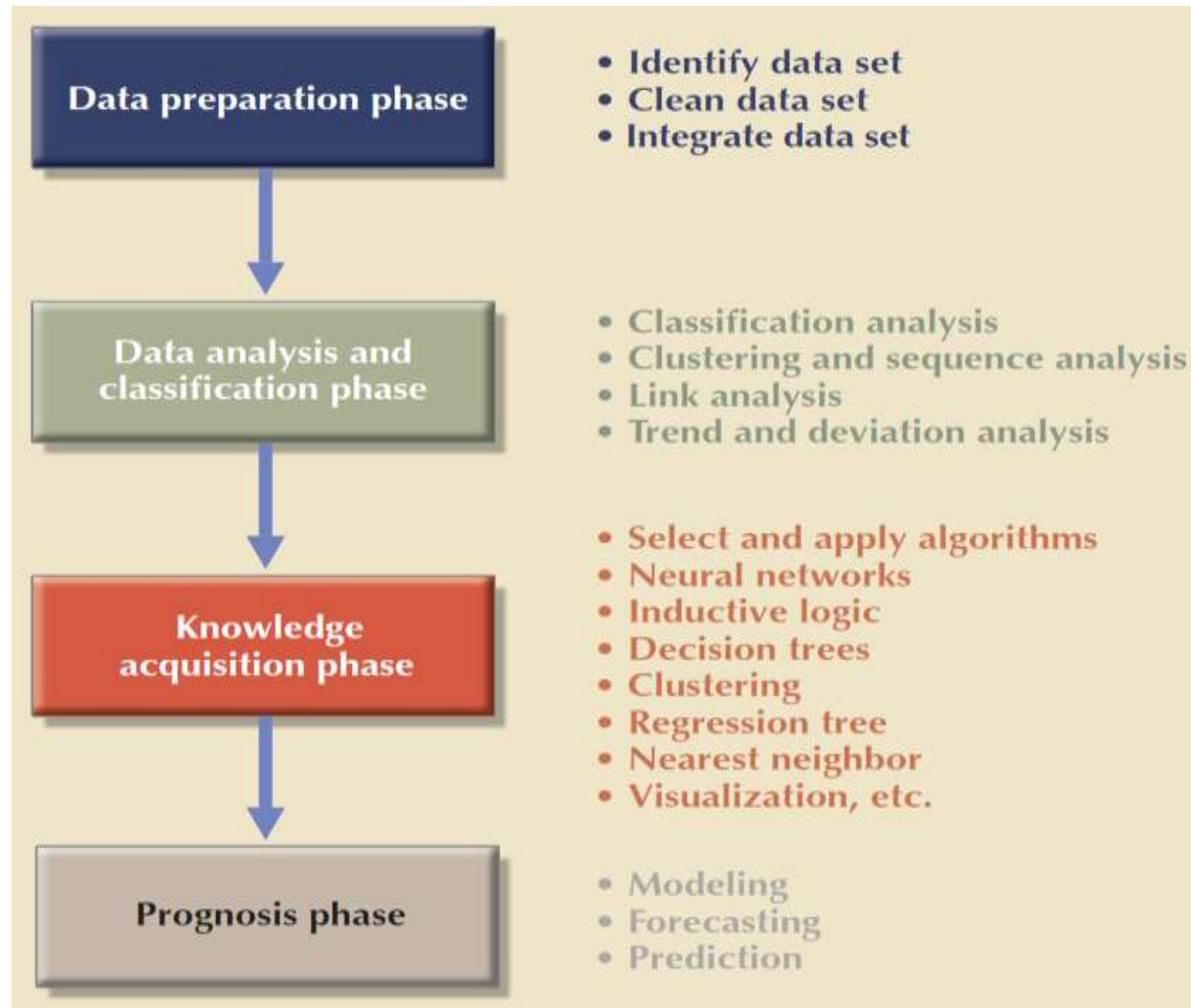
Data Analytics

- Encompasses a wide range of mathematical, statistical, and modeling techniques to extract knowledge from data
 - Subset of BI functionality
- Classification of tools
 - **Explanatory analytics:** Focuses on discovering and explaining data characteristics and relationships based on existing data
 - **Predictive analytics:** Focuses on predicting future outcomes with a high degree of accuracy

Data Mining

- Analyzing massive amounts of data to:
 - Uncover hidden trends, patterns, and relationships
 - Form computer models to stimulate and explain the findings
 - Use the models to support business decision making
- Run in two modes
 - Guided
 - Automated

Figure 13.16 - Data-Mining Phases



Predictive Analytics

- Employs mathematical and statistical algorithms, neural networks, artificial intelligence, and other advanced modeling tools
- Creates actionable predictive models based on available data
 - Next logical step after data mining
- Adds value to an organization
 - Helps optimize the existing processes
 - Identify hidden problems
 - Anticipate future problems or opportunities

Database systems

ITCS2322

Chapter 14 : Big Data and NoSQL

اعداد : دكتور محمد عبد اللطيف راضي

Learning Objectives

- After completing this chapter, you will be able to:
- Explain the role of Big Data in modern business
- Describe the primary characteristics of Big Data and how these go beyond the traditional “3 Vs”
- Explain how the core components of the Hadoop framework operate
- Identify the major components of the Hadoop ecosystem
- Summarize the four major approaches of the NoSQL data model and how they differ from the relational model
- Describe the characteristics of NewSQL databases
- Understand how to work with document databases using MongoDB
- Understand how to work with graph databases using Neo4j

Big Data

Volume: the quantity of data to be stored

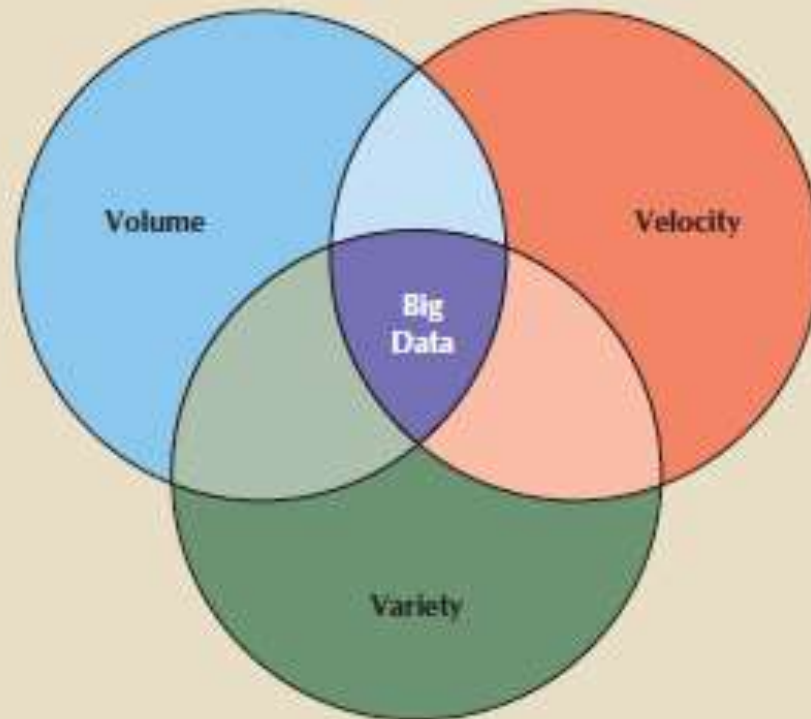
Velocity: the speed at which data is entering the system

Variety: the variations in the structure of the data to be stored

- That makes the data unsuitable for management by a relational database management system.

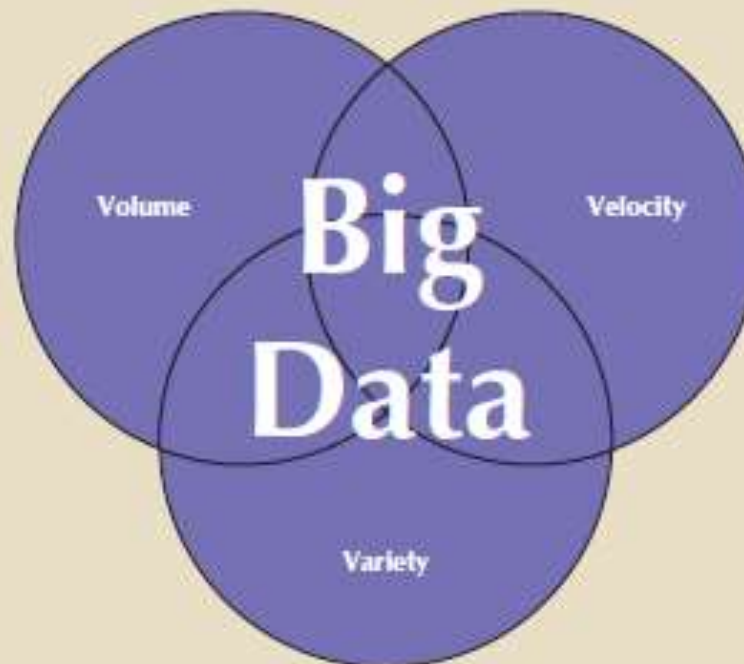
Big Data

FIGURE 14.1 ORIGINAL VIEW OF BIG DATA



Big Data

FIGURE 14.2 CURRENT VIEW OF BIG DATA



Big Data- Volume

- Volume, the quantity of data to be stored, is a key characteristic of Big Data.
- The storage capacities associated with Big Data are extremely large.
- as the quantity of data needing to be stored increases, the need for larger storage devices increases as well.
- **Scaling up** is keeping the same number of systems, but migrating each system to
 - a larger system.
- **scaling out** means that when the workload exceeds the capacity of a server, the workload is spread out across a number of servers.
 - This is also referred to as *clustering*

Big Data- Velocity

- The rate at which new data enters the system as well as the rate at which the data must be processed.
- In addition for Big Data to be actionable, that data must be processed at a very rapid pace.
- The velocity of processing can be broken down into two categories.
 - Stream processing
 - Feedback loop processing

Big Data- Velocity

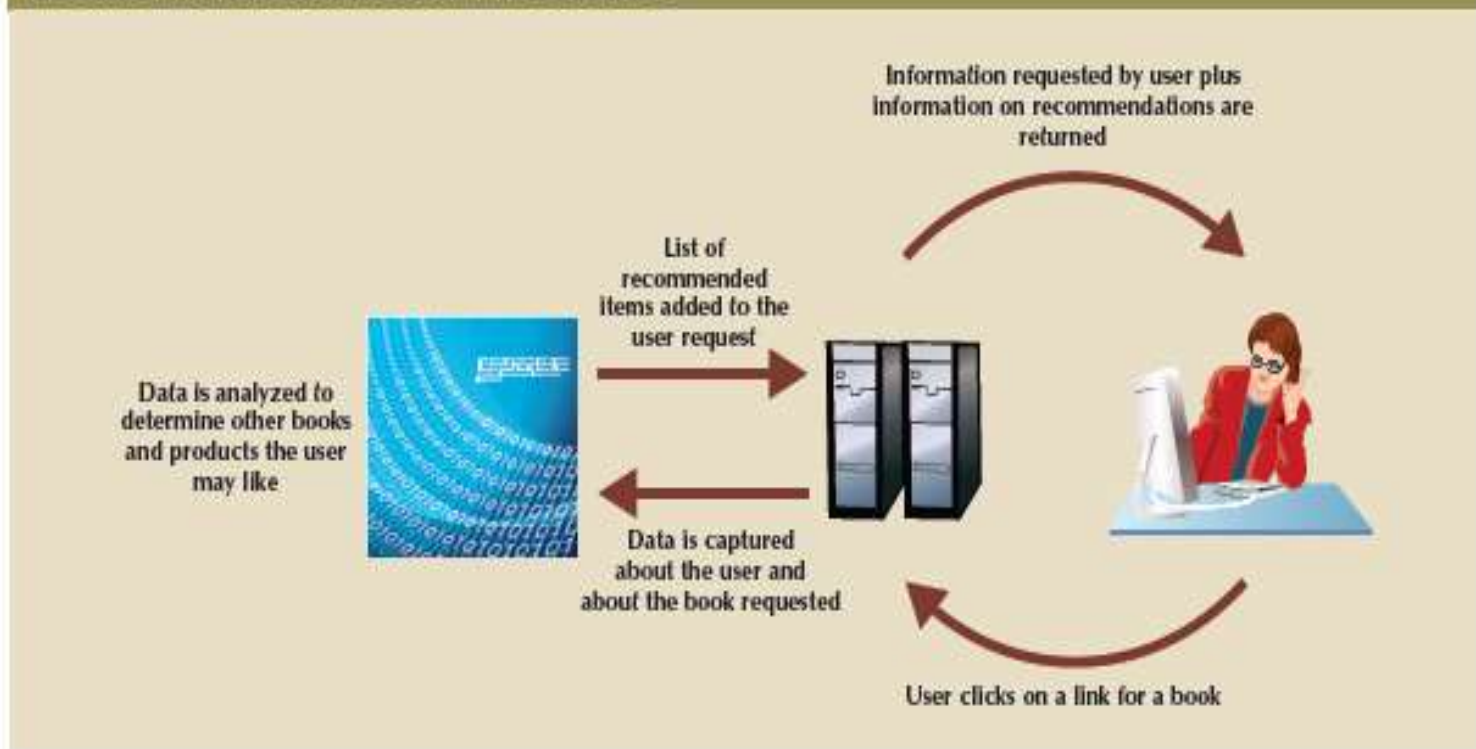
- **Stream processing** focuses on input processing, and it requires analysis of the data stream as it enters the system.
- In some situations, large volumes of data can enter the system at such a rapid pace that it is not feasible to try to store all of the data.
- The data must be processed and filtered as it enters the system to determine which data to keep and which data to discard.

Big Data- Velocity

- **Feedback loop processing** refers to the analysis of the data to produce actionable results.
- While stream processing could be thought of as focused on inputs, feedback loop processing can be thought of as focused on outputs.
- The process of capturing the data, processing it into usable information, and then acting on that information is a feedback loop.

Big Data- Velocity

FIGURE 14.3 FEEDBACK LOOP PROCESSING



Big Data- Variety

- Variety refers to the vast array of formats and structures in which the data may be captured.
- Data can be considered to be structured, unstructured, or semistructured.
- **Structured data** is data that has been organized to fit a predefined data model. Relational databases
- **Unstructured data** is data that is not organized to fit into a predefined data model.
 - maps, satellite images, emails, texts, tweets, videos, transcripts, and a whole host of other data forms.
- **Semistructured data** combines elements of both some parts of the data fit a predefined model while other parts do not.

Big Data- Other Characteristics

TABLE 14.2

ADDITIONAL Vs OF BIG DATA

CHARACTERISTIC	DESCRIPTION
Variability	Data meaning changes based on context.
Veracity	Data is correct.
Value (Viability)	Data can provide meaningful information.
Visualization	Data can be presented in such a way as to make it understandable.

Big Data- Other Characteristics

- **Variability** refers to the changes in the meaning of the data based on context.
 - Variety is about differences in structure.
 - Variability is about differences in meaning. Variability is especially relevant in areas such as sentiment analysis that attempt to understand the meanings of words

Big Data- Other Characteristics

- **Veracity** refers to the trustworthiness of the data.
- Can decision makers reasonably rely on the accuracy of the data and the information generated from it?
- This is especially pertinent given the automation of data capture and some of the analysis. Uncertainty about the data can arise from several causes,
 - having to capture only selected portions of the data due to high velocity.
 - customers' opinions and preferences can change over time, so comments at one point in time might not be suitable for action at another point in time.

Big Data- Other Characteristics

- **Value**, also called *viability*, refers to the degree to which the data can be analyzed to provide meaningful information that can add value to the organization.
- Just because a set of data *can* be captured does not mean that it *should* be captured.
- Only data that can form the basis for analysis that has the potential to impact organizational behavior should be included in a company's Big Data efforts.

Big Data- Other Characteristics

- **Visualization** is the ability to graphically present the data in such a way as to make it understandable.
- Volumes of data can leave decision makers awash in facts but with little understanding of what the facts mean.
- Visualization is a way of presenting the facts so that decision makers can comprehend the meaning of the information to gain insights.

Hadoop

- Big Data requires a different approach to distributed data storage that is designed for large-scale clusters.
- Hadoop has become the de facto standard for most Big Data storage and processing.
- Hadoop is a Java-based framework for distributing and processing very large data sets across clusters of computers.
 - Hadoop Distributed File System (HDFS) and
 - MapReduce.

Hadoop Distributed File System (HDFS)

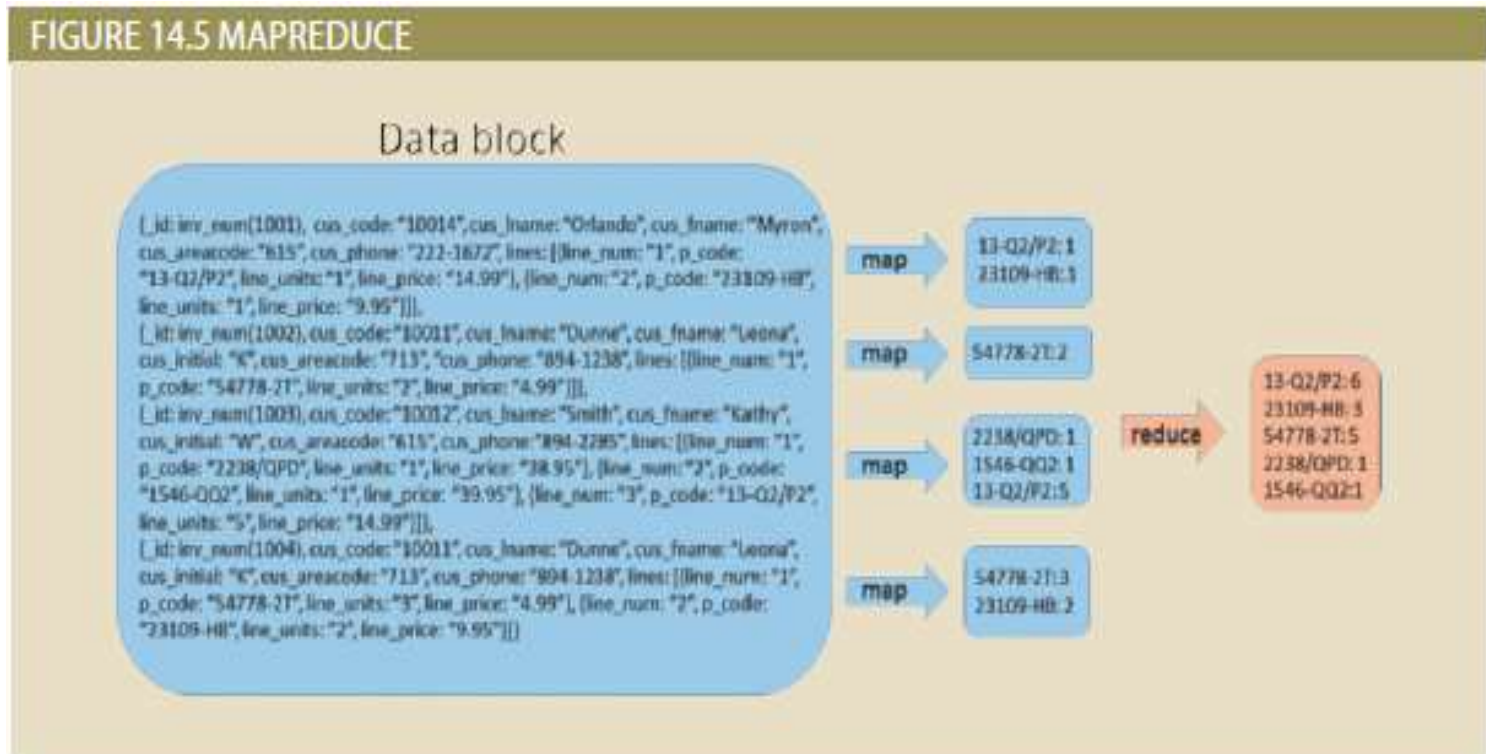
- The Hadoop Distributed File System (HDFS) approach to distributing data is based on several key assumptions:
- *High volume*
- *Write-once, read-many*
- *Streaming access*
 - Hadoop is optimized for batch processing of entire files as a continuous stream of data.
- *Fault tolerance*
 - Hadoop is designed to be distributed across thousands of low-cost, commodity computers.
 - HDFS is designed to replicate data across many different devices so that when one device fails, the data is still available from another device

MapReduce

- Is the computing framework used to process large data sets across clusters.
- MapReduce follows the principle of *divide and conquer*.
 - Takes a complex task,
 - Breaks it down into a collection of smaller subtasks,
 - Performs the subtasks all at the same time, and
 - Combines the result of each subtask to produce a final result for the original task.

MapReduce

FIGURE 14.5 MAPREDUCE



NoSQL

- NoSQL is array of nonrelational database technologies that have developed to address the challenges represented by Big Data.
- NoSQL could stand for “Not Only SQL.”
 - Keyvalue data stores,
 - Document databases,
 - Column-oriented databases, and
 - Graph databases

NoSQL

TABLE 14.3

NoSQL DATABASES

NoSQL CATEGORY	EXAMPLE DATABASES	DEVELOPER
Key-value database	Dynamo Riak Redis Voldemort	Amazon Basho Redis Labs LinkedIn
Document databases	MongoDB CouchDB OrientDB RavenDB	MongoDB, Inc. Apache OrientDB Ltd. Hibernate Rhinos
Column-oriented databases	HBase Cassandra Hypertable	Apache Apache (originally Facebook) Hypertable, Inc.
Graph databases	Neo4J ArangoDB GraphBase	Neo4j ArangoDB, LLC FactNexus

Key-Value Databases

- Stores data as a collection of key-value pairs.
- The key acts as an identifier for the value.
- The value can be anything such as text, an XML document, or an image.
- The database does not understand the contents of the value component or its meaning
- the database stores whatever value is provided for the key.
- It is the job of the applications that use the data to understand the meaning of the data in the value component

Key-Value Databases

- Key-value pairs are typically organized into “buckets.”
- A bucket can roughly be thought of as the KV database equivalent of a table.
- A bucket is a logical grouping of keys.
- Key values must be unique within a bucket, but they can be duplicated across buckets.
- All data operations are based on the bucket plus the key.

Key-Value Databases

FIGURE 14.7 KEY-VALUE DATABASE STORAGE

Bucket = Customer

Key	Value
10010	"LName Ramas FName Alfred Initial A Areacode 615 Phone 844-2573 Balance 0"
10011	"LName Dunne FName Leona Initial K Areacode 713 Phone 894-1238 Balance 0"
10014	"LName Orlando FName Myron Areacode 615 Phone 222-1672 Balance 0"

Document Databases

- Conceptually similar to key-value databases, and they can almost be considered a subtype of KV databases.
- A document database is a NoSQL database that stores data in tagged documents in key-value pairs.
- Unlike a KV database where the value component can contain any type of data, a document database always stores a document in the value component.
- The document can be in any encoded format, such as XML, JSON (JavaScript Object Notation), or BSON (Binary JSON).
- document databases attempt to understand the content of the value component.

- Document databases group documents into logical groups called **collections**.
- Document may be retrieved by specifying the collection and key.

FIGURE 14.8 DOCUMENT DATABASE TAGGED FORMAT

Collection = Customer

Key	Document
10010	{LName: "Ramas", FName: "Alfred", Initial: "A", Areacode: "615", Phone: "844-2573", Balance: "0"}
10011	{LName: "Dunne", FName: "Leona", Initial: "K", Areacode: "713", Phone: "894-1238", Balance: "0"}
10014	{LName: "Orlando", FName: "Myron", Areacode: "615", Phone: "222-1672", Balance: "0"}

Column-Oriented Databases

- column-oriented database or columnar database can refer to traditional, relational database technologies that use column-centric storage instead of row-centric storage.

Column-Oriented Databases

FIGURE 14.9 COMPARISON OF ROW-CENTRIC AND COLUMN-CENTRIC STORAGE

CUSTOMER relational table

Cus_Code	Cus_LName	Cus_FName	Cus_City	Cus_State
10010	Ramas	Alfred	Nashville	TN
10011	Dunne	Leona	Miami	FL
10012	Smith	Kathy	Boston	MA
10013	Olowski	Paul	Nashville	TN
10014	Orlando	Myron		
10015	O'Brian	Amy	Miami	FL
10016	Brown	James		
10017	Williams	George	Mobile	AL
10018	Farriss	Anne	Opp	AL
10019	Smith	Olette	Nashville	TN

Row-centric storage

Block 1 10010,Ramas,Alfred,Nashville,TN 10011,Dunne,Leona,Miami,FL	Block 4 10016,Brown,James,NULL,NULL 10017,Williams,George,Mobile,AL
Block 2 10012,Smith,Kathy,Boston,MA 10013,Olowski,Paul,Nashville,TN	Block 5 10018,Farriss,Anne,OPP,AL 10019,Smith,Olette,Nashville,TN
Block 3 10014,Orlando,Myron,NULL,NULL 10015,O'Brian,Amy,Miami,FL	

Column-centric storage

Block 1 10010,10011,10012,10013,10014 10015,10016,10017,10018,10019	Block 4 Nashville,Miami,Boston,Nashville,NULL Miami,NULL,Mobile,Opp,Nashville
Block 2 Ramas,Dunne,Smith,Olowski,Orlando O'Brian,Brown,Williams,Farriss,Smith	Block 5 TN,FL,MA,TN,NULL, FL,NULL,AL,AL,TN
Block 3 Alfred,Leona,Kathy,Paul,Myron Amy,James,George,Anne,Olette	

Graph Databases

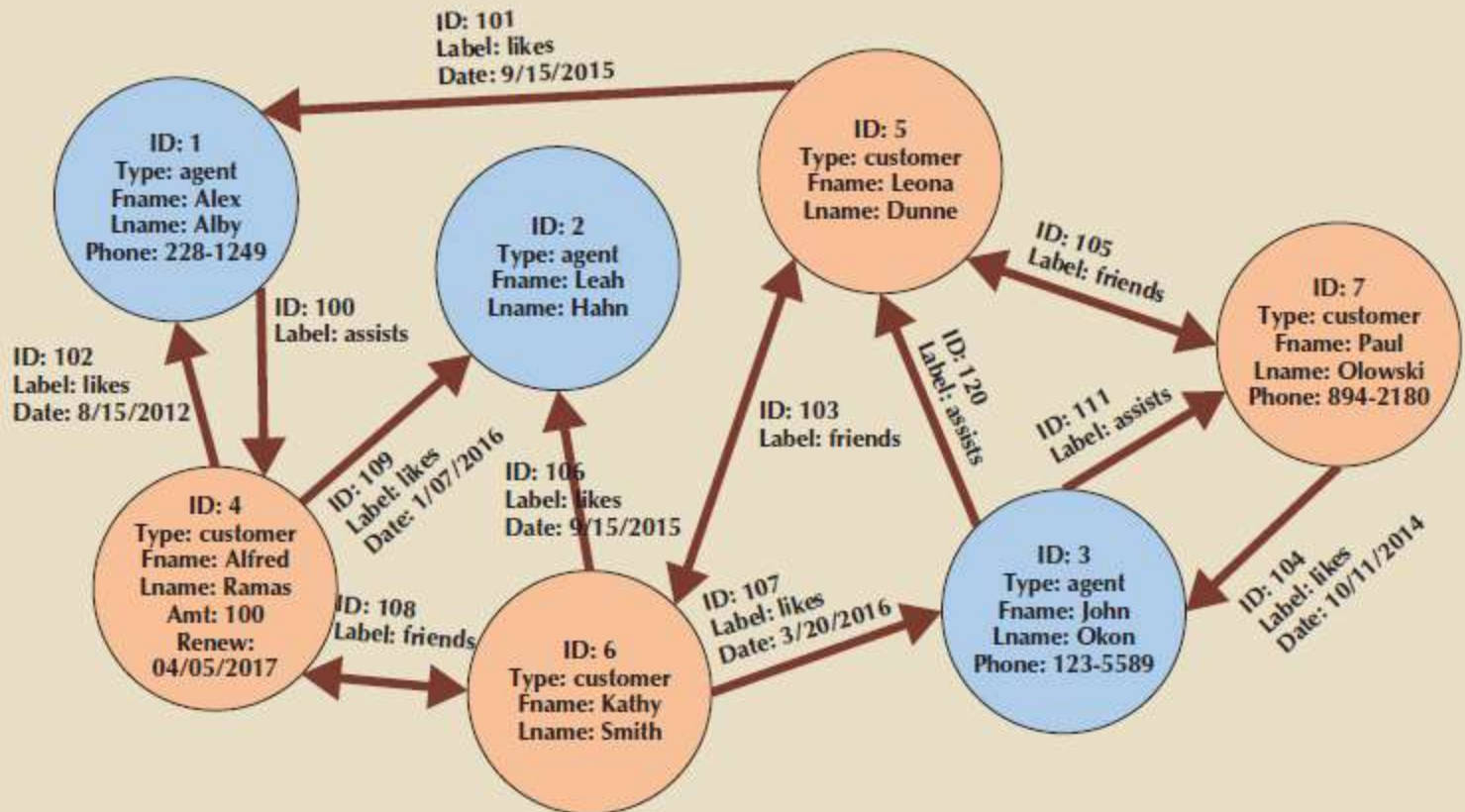
- A graph database is a NoSQL database based on graph theory to store data about relationship-rich environments.
- Graph theory is a mathematical and computer science field that models relationships, or *edges*, between objects called nodes.
- Modeling and storing data about relationships is the focus of graph databases

Graph Databases

- Interest in graph databases originated in the area of social networks.
- The primary components of graph databases are nodes, edges, and properties.
- The **node** is a specific instance of something we want to keep data about.
- An **edge** is a relationship between nodes.
- Note that edges can also have **properties**.

Graph Databases

FIGURE 14.11 GRAPH DATABASE REPRESENTATION



Working with Document Databases Using **MongoDB**

- A simple JSON document to store data on a book might look like this:
- `{_id: 101, title: 'Database Systems'}`
- This document contains two *key:value* pairs:
 - *_id* is a key with 101 as the associated value
 - *title* is a key with 'Database Systems' as the associated value
- `{_id: 101, title: 'Database Systems', author: ['Coronel', 'Morris']}`

Working with Document Databases Using **MongoDB**

```
{  
  _id: 101,  
  title: 'Database Systems',  
  author: ['Coronel', 'Morris']  
}
```

Working with Graph Databases Using Neo4j

- CREATE (:Member {mid: 1, fname: "Phillip", lname: "Stallings"})
- To retrieve our single member node.
 - MATCH (m)
 - RETURN(m)
- to retrieve Phillip Stallings
 - MATCH (m {fname: "Phillip"}), (x {lname: "Stallings"})
 - RETURN m

:Scenario

:A hospital chain called "HealthyLife Hospitals" operates across three regions: East, West, and Central. The hospital management wants a data mart to monitor patient care and resource utilization. The system needs to track

- .Patient admissions and discharges across departments (e.g., Cardiology, Orthopedics)
- .Bed occupancy rates and availability
- .Performance of doctors and nurses (e.g., patients treated, surgeries performed)
- .Patient demographics, including age group, gender, and insurance status
- .Hospital revenue from treatments, surgeries, and diagnostics

?Which of the following are suitable **dimensions** for analyzing hospital resource utilization
(Select **three** correct answers)

Region (East, West, Central) .1 ☐

Time (Day, Month, Year) .2 ☐

Department (e.g., Cardiology, Orthopedics) .3 ☐

Bed Occupancy Rate .4 ☐

Doctor Role (e.g., Surgeon, General Physician) .5 ☐

:Fault tolerance in Hadoop ensures

Automatic database normalization .a ☐

Dynamic schema validation .b ☐

Efficient real-time analytics .c ☐

High availability despite hardware failures .d ☐

?What is the relationship between a fact table and a dimension table in a star schema

One-to-many .a ☐

One-to-one .b ☐

Many-to-one .c ☐

Many-to-many .d ☐

.Data warehouses are primarily designed to support transactional processing systems

True .a ☐

False .b ☐

"Velocity" in Big Data refers to the trustworthiness of the data"

False .a ☐

True .b ☐

?Which of the following best describes cost-based optimization .11

.It is based solely on user-defined hints .a ☐

.It uses statistical data to determine the most efficient execution plan .b ☐

.It allows the DBMS to execute queries without optimization .c ☐

.It relies on preset rules to execute queries .d ☐

?What is a major factor in query processing bottlenecks .8

Efficient use of indexes .a ☐

Minimizing network latency .b ☐

Use of a rule-based optimizer .c ☐

Insufficient RAM and CPU resources .d ☐

?Document databases store data in which format

Key-value pairs only .a ☐

Tables .b ☐

Rows and columns .c ☐

Tagged documents like JSON or XML .d ☒

5. In the context of concurrency control, what is 'serializability'?

- a. The use of timestamps to control transaction order ☐
- b. The ability to run multiple transactions at the same time ☐
- c. The process of locking database tables ☐
- d. Ensuring that the execution of concurrent transactions results in the same state as serial execution ☐

Arrange the steps to create an Oracle Apex workspace for free:

First Step:

Second Step:

Third Step:

Fourth Step:

Fifth Step:

Sixth Step:

Seventh Step:

Eighth Step:

Change Password	Click Create Workspace	Go to Oracle Apex website	Login with a Workspace name and Email
Start for Free Today	Request a workspace	Check Your entered Email to Create a Workspace	Choose Free APEX Workspace

Performance transparency allows a DDBMS to function as if it were centralized, hiding distributed complexity

False .a ☐

True .b ☐

?Which type of key is typically found in a fact table

Composite primary key .a ☐

Surrogate key .b ☐

Alternate key .c ☐

Unique key .d ☐

7. Which of these types of indexes is primarily used in data warehouses with repetitive column values?

a. Primary index ☐

b. Bitmap index ☐

c. Hash index ☐

d. B-tree index ☐

```
projects /insert 5
  name /nn
  owner
  milestones /insert 10
    name /nn
    status /check open completed closed /values open, closed, completed
    owner
    started date
    closed date
  links /insert 5
    name /nn
    url
  attachments
    contributed by
    attachment file
  action items /insert 12
    action
    desc clob
    owner
    status /check open completed closed

view project_ms projects milestones
view project_ai projects action_items
```

Depending on the previous SQL CODE, choose all applicable Foreign keys.

- project_id number constraint milestones_project_id_fk references projects, .1 ☐
- assignee_id number constraint todos_assignee_id_fk references assignees, .2 ☐
- project_id number constraint attachments_project_id_fk references projects, .3 ☐
- project_id number constraint links_project_id_fk references projects, .4 ☐

?What is a star schema

A key performance indicator .a ☐

A form of data mining .b ☐

A type of database schema used in data warehouses .c ☐

A method of data visualization .d ☐

.In DDBMS, data is always stored at a single site

False .a ☐

True .b ☐

?What does "scaling out" mean in Big Data

- Migrating to larger systems .a ☐
- Reducing server count for efficiency .b ☐
- Adding more servers to distribute workload .c ☐
- Increasing storage device capacity .d ☐

?What does the SQL Execution Phase involve

Validating the syntax of the query .a ☐

Returning the results to the client .b ☐

Executing the access plan and retrieving data .c ☐

Transforming the query into an optimized version .d ☐

?Which transparency allows users to access distributed data without knowing its location

Transaction transparency .a ☐

Data distribution .b ☐

Location transparency .c ☐

Fragmentation transparency .d ☐

You have the following Quick SQL code:

```
projects /insert 5
  name /nn
  owner
  milestones /insert 10
    name /nn
    status /check open completed closed /values open, closed, completed
    owner
    started date
    closed date
  links /insert 5
    name /nn
    url
  attachments
    contributed by
    attachment file
  action items /insert 12
    action
    desc clob
    owner
    status /check open completed closed

view project_ms projects milestones
view project_ai projects action_items
```

Depending on the previous SQL CODE, choose all applicable Foreign keys.

project_id number constraint milestones_project_id_fk references projects, .1 ☒

assignee_id number constraint todos_assignee_id_fk references assignees, .2 ☒

project_id number constraint attachments_project_id_fk references projects, .3 ☒

project_id number constraint links_project_id_fk references projects, .4 ☐

:In a key-value database, data operations are based on

Queries written in SQL .a ☐

Keys within a bucket .b ☐

Row identifiers .c ☐

Relational table indices .d ☐

?In database recovery, if a transaction is rolled back after a checkpoint, what should be done .15

Commit the transaction .a ☐

Do nothing .b ☐

Ignore the transaction .c ☐

Redo the transaction .d ☐

.The SQL fetching phase includes executing all I/O operations and applying locks

False .a ☐

True .b ☐

You have the following Quick SQL code:

```
projects /insert 5
  name /nn
  owner
  milestones /insert 10
    name /nn
    status /check open completed closed /values open, closed, completed
    owner
    started date
    closed date
  links /insert 5
    name /nn
    url
  attachments
    contributed by
    attachment file
  action items /insert 12
    action
    desc clob
    owner
    status /check open completed closed

view project_ms projects milestones
view project_ai projects action_items
```

Depending on the previous SQL CODE, it will create tables.

6 .1 ☐

3 .2 ☐

4 .3 ☐

5 .4 ☐

?What is the purpose of MapReduce in Hadoop

To process and analyze large data sets by dividing tasks .a ☐

To visualize Big Data .b ☐

To store data in a column-oriented fashion .c ☐

To manage key-value pairs .d ☐

:Operational data is best suited for

Supporting daily business transactions .a ☐

Time-series analysis .b ☐

Aggregated historical analysis .c ☐

Predictive analytics .d ☐

```
projects /insert 5
  name /nn
  owner
  milestones /insert 10
    name /nn
    status /check open completed closed /values open, closed, completed
    owner
    started date
    closed date
  links /insert 5
    name /nn
    url
  attachments
    contributed by
    attachment file
  action items /insert 12
    action
    desc clob
    owner
    status /check open completed closed

view project_ms projects milestones
view project_ai projects action_items
```

Depending on the previous SQL CODE, it will create tables.

6 .1 ☐

3 .2 ☐

4 .3 ☐

5 .4 ☐

DimDoctor

DimDepartment

DimHospital

DimBed

DimDate

Fact_Bed_Utilization

FactPatientCare

.Concurrency control helps prevent multiple users from accessing the same data at the same time

True .a ☐

False .b ☐

.Indexes are always beneficial for query optimization, regardless of the data sparsity of the indexed column

True .a ☐

False .b ☐

Scenario:

A hospital chain called "HealthyLife Hospitals" operates across three regions: East, West, and Central. The hospital management wants a data mart to monitor patient care and resource utilization. The system needs to track:

Patient admissions and discharges across departments (e.g., Cardiology, Orthopedics).

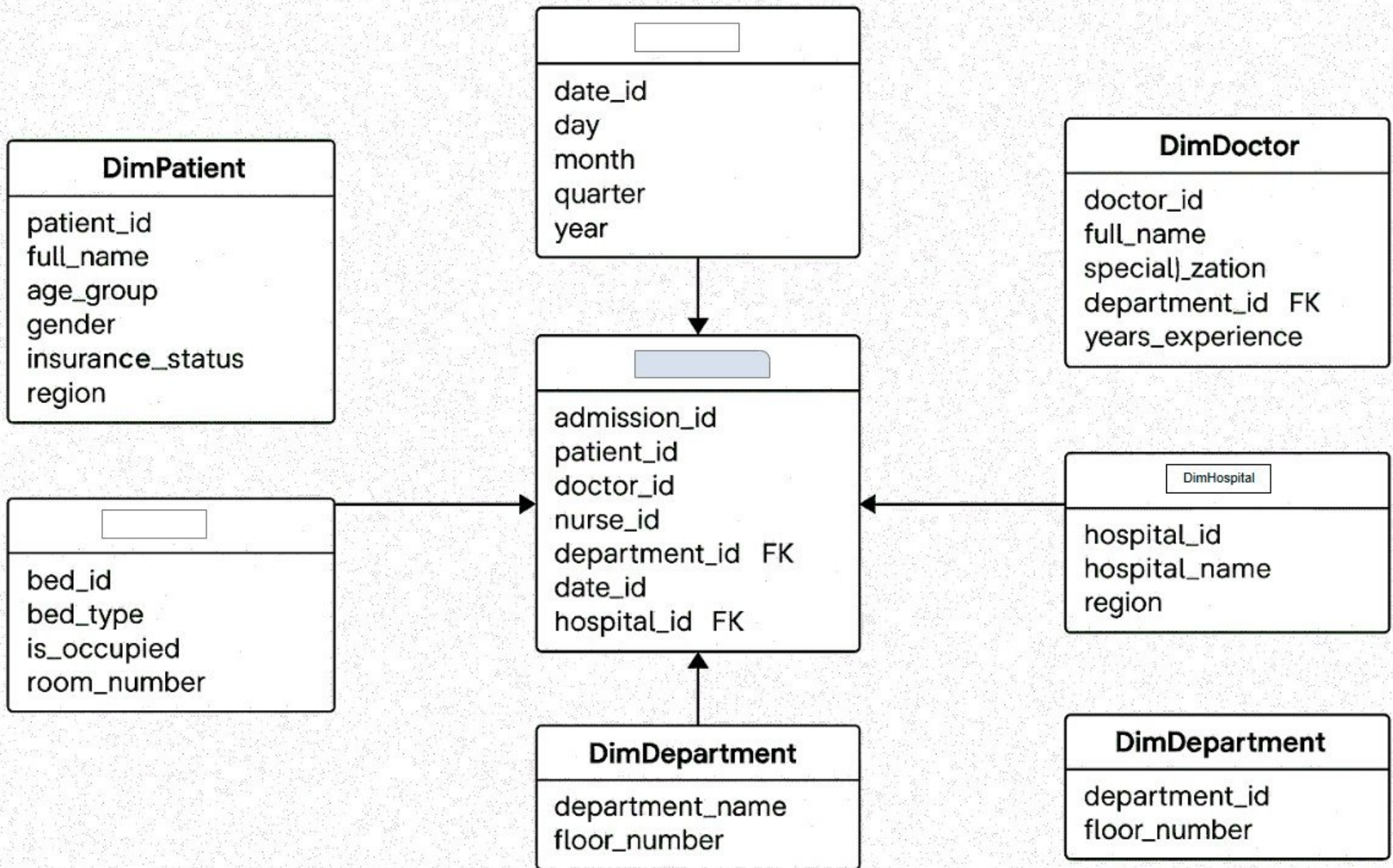
Bed occupancy rates and availability.

Performance of doctors and nurses (e.g., patients treated, surgeries performed).

Patient demographics, including age group, gender, and insurance status.

Hospital revenue from treatments, surgeries, and diagnostics.

Drag and drop the dimensions and Facts in the right place.



HealthyLife Hospitals

.A deadlock occurs when multiple transactions are waiting indefinitely for resources held by each other

True .a ☐

False .b ☐

.Indexes are always beneficial for query optimization, regardless of the data sparsity of the indexed column

True .a ☐

False .b ☐

?Which of the following visualizations would be best for displaying trends over time

Scatter plot .a ☐

Line graph .b ☐

Pie chart .c ☐

Heat map .d ☐

:Scenario

:A hospital chain called "HealthyLife Hospitals" operates across three regions: East, West, and Central. The hospital management wants a data mart to monitor patient care and resource utilization. The system needs to track

- .Patient admissions and discharges across departments (e.g., Cardiology, Orthopedics)
- .Bed occupancy rates and availability
- .Performance of doctors and nurses (e.g., patients treated, surgeries performed)
- .Patient demographics, including age group, gender, and insurance status
- .Hospital revenue from treatments, surgeries, and diagnostics

?Which of the following are appropriate KPIs for measuring **hospital resource utilization**
(Select **three** correct answers)

- Insurance Status of Patients .1 ☐
- Number of Surgeries Performed .2 ☐
- Patient Readmission Rate .3 ☐
- Revenue per Department .4 ☐
- Percentage of Bed Availability .5 ☐

?What is a system that manages data storage and processing across interconnected sites

Centralized Database Management System .a ☐

Local Database Management System .b ☐

Distributed Database Management System .c ☐

Cloud Database Management System .d ☐

?Which of the following describes decision support data

- Current data for immediate use .a ☐
- Data optimized for analysis and decision support tasks .b ☐
- Highly normalized data for real-time operations .c ☐
- Data from a single source only .d ☐

:Veracity in Big Data ensures

- High velocity of data ingestion .a ☐
- Only real-time data processing .b ☐
- Low variability in data structure .c ☐
- High trust in data accuracy .d ☐

.Big Data's "velocity" refers to the variety of data types being processed

True .a ☐

False .b ☐

?Which level of lock granularity provides the highest level of concurrency

Database-level lock .a ☐

Page-level lock .b ☐

Row-level lock .c ☐

Table-level lock .d ☐

?Which recovery technique uses the transaction log to "redo" transactions committed after the last checkpoint

Optimistic Approach .a ☐

Deferred-Write .b ☐

Write-Through .c ☐

Two-Phase Locking .d ☐

?What is location transparency in a distributed database

Processing of all transactions on a single site .a ☐

Allowing users to access data without worrying about fragment location .b ☐

Storing data on local systems only .c ☐

Guaranteeing data consistency across all sites .d ☐

?Which of the following is NOT a characteristic of Big Data

Validation .a ☐

Velocity .b ☐

Volume .c ☐

Variety .d ☐

?Which of the following is a mode in which data mining can run

Supervised and unsupervised .a ☐

Explanatory and predictive .b ☐

Historical and futuristic .c ☐

Automated and guided .d ☐

?Which NoSQL database model uses nodes and edges to represent relationships

Column-oriented databases .a ☐

Key-value databases .b ☐

Graph databases .c ☐

Document databases .d ☐

?Which phase is NOT part of the general phases of data mining

Prognosis .a ☐

Data preparation .b ☐

Knowledge acquisition .c ☐

Trend forecasting .d ☐

?Which storage optimization strategy involves separating high-usage tables into distinct storage volumes?

- Table space optimization .a ☐
- Data cache allocation .b ☐
- Disk partitioning .c ☐
- Creating temporary table spaces .d ☐

?Which of the following is NOT a characteristic of a dimension

Descriptive characteristics .a ☐

Aggregated measures .b ☐

Hierarchies for drill-down analysis .c ☐

Qualifying attributes .d ☐

.MapReduce processes data by splitting it into smaller tasks

True .a ☐

False .b ☐

:Scenario

:A hospital chain called "HealthyLife Hospitals" operates across three regions: East, West, and Central. The hospital management wants a data mart to monitor patient care and resource utilization. The system needs to track

- .Patient admissions and discharges across departments (e.g., Cardiology, Orthopedics)
- .Bed occupancy rates and availability
- .Performance of doctors and nurses (e.g., patients treated, surgeries performed)
- .Patient demographics, including age group, gender, and insurance status
- .Hospital revenue from treatments, surgeries, and diagnostics

?Which of the following is most likely to be included as a measure in the **Fact Table** for HealthyLife Hospitals

Patient Age Group .1 ☐

Number of Surgeries Performed .2 ☐

Region (East, West, Central) .3 ☐

Department Name .4 ☐

?What is the purpose of the transaction log in a DBMS .4

To ensure faster execution of queries .a ☐

To store all SQL queries run by users .b ☐

To lock database tables during a transaction .c ☐

To track all transactions that update the database .d ☐

?Which of the following is an example of a BI technology trend

- Static pie charts .a ☐
- Paper-based reporting .b ☐
- Relational database optimization .c ☐
- Cloud-based and mobile BI systems .d ☐

?What is the primary purpose of a star schema in data modeling

- To map relational data into object-oriented databases .a ☐
- To represent multidimensional data in a relational database .b ☐
- To simplify operational database queries .c ☐
- To ensure data encryption .d ☐

.Distributed query processing is one of C.J. Date's twelve commandments for DDBMS

True .a ☐

False .b ☐

:Key performance indicators (KPIs) measure

- Business performance in meeting objectives .a ☐
- Storage costs of a data warehouse .b ☐
- Customer satisfaction levels only .c ☐
- The effectiveness of employees .d ☐

?MongoDB is an example of which type of database

Column-oriented database .a ☐

Document database .b ☐

Key-value database .c ☐

Relational database .d ☐

:In Hadoop, fault tolerance is achieved through

Data replication across multiple devices .a ☐

A centralized server system .b ☐

Real-time data processing .c ☐

Using only high-performance computers .d ☐